

LAB

Evaluate the performance of an AI agent using MLflow

Contents

Introduction	2
Software requirements	2
Objective	2
Lab steps	2
Estimated duration to complete	2
Scenario	2
Background information	2
Challenge.....	3
Solutions	3
Create a GitHub account.....	3
Overview.....	3
Instructions	3
Create a Replicate account	8
Overview.....	8
Instructions	9
Sign up for Google Colab	15
Overview.....	15
Instructions	15
Initialize the AI agent and tools	19
Overview.....	19
Instructions	19
Evaluate the trajectory and final response of the AI agent	28
Overview.....	28
Instructions	28
Conclusion.....	31

Introduction

In this lab, you'll assume the role of an AI specialist to evaluate a pilot version of an AI agent using MLflow.

Software requirements

To complete this lab, you'll need access to a Replicate account, which allows you to use artificial intelligence (AI) models to perform tasks. You'll also need a Replicate application programming interface (API) token, which acts as a secure key to connect your Colab environment so that the lab can run smoothly.

Basic familiarity with Python will enable you to better follow how the agent works and its tool usage.

Objective

After completing this lab, you should be able to:

- Evaluate the performance of an AI agent using MLflow

Lab steps

This lab requires you to complete the following procedures:

- Create a GitHub account
- Create a Replicate account
- Sign up for Google Colab
- Initialize the AI agent and tools
- Evaluate the trajectory and final response of the AI agent

Estimated duration to complete

30 minutes

Scenario

Background information

TechMart, a fast-growing e-commerce platform, is transforming its shopping experience with an AI agent built on IBM Granite. The agent replaces static catalogs, offering real-time recommendations, answering questions, and personalizing the customer's journey.

You are part of TechMart's newly formed AI evaluation team, serving as the AI specialist for this project. Your role is to assess the agent's performance by evaluating product recommendations, response accuracy, and the overall user experience.

Challenge

TechMart users often leave the site when the AI agent fails to provide relevant product recommendations or assist effectively with catalog navigation. Additionally, missing or inconsistent details about products, pricing, or stock levels can erode the user's trust.

Solutions

To support this, your team uses MLflow to track, evaluate, and visualize the agent's performance. In this lab, you'll determine whether the AI agent is ready for full-scale deployment.

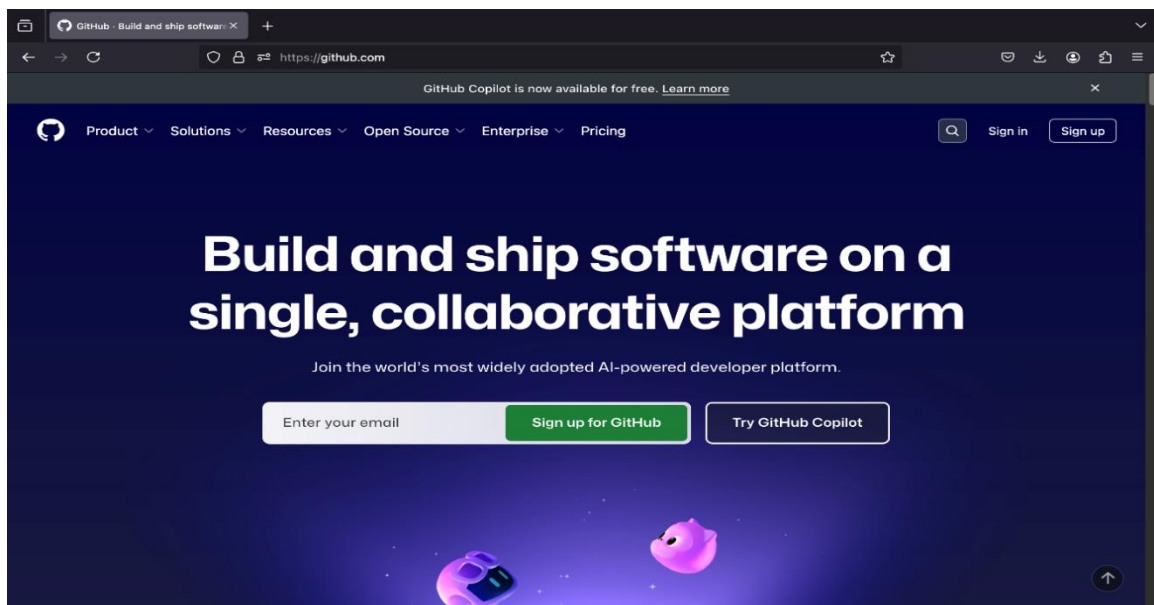
Create a GitHub account

Overview

In this procedure, you'll set up a GitHub account. GitHub is a platform that helps developers store, manage, and share code, while also supporting collaboration through tools such as version control, bug tracking, and task management. Setting up a GitHub account provides access to the Replicate platform, which is required to complete the lab efficiently.

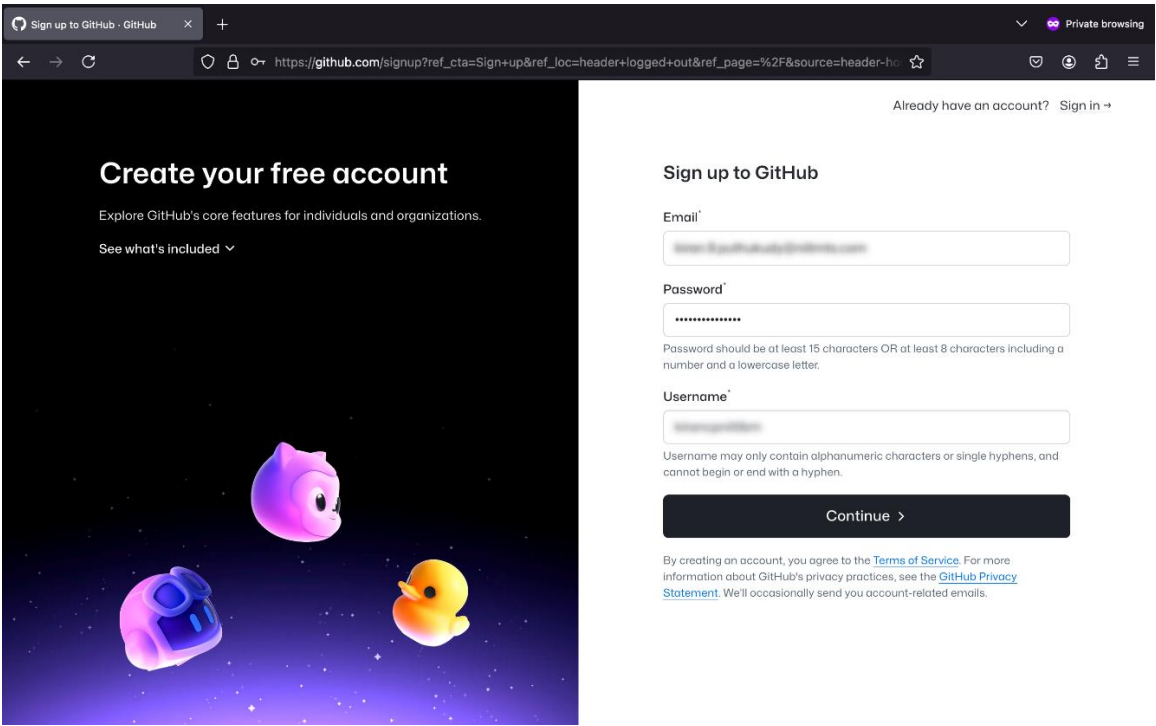
Instructions

1. To create a GitHub account, go to the [GitHub](https://github.com) website.
2. Select the **Sign up for GitHub** button, as shown in the following screenshot.

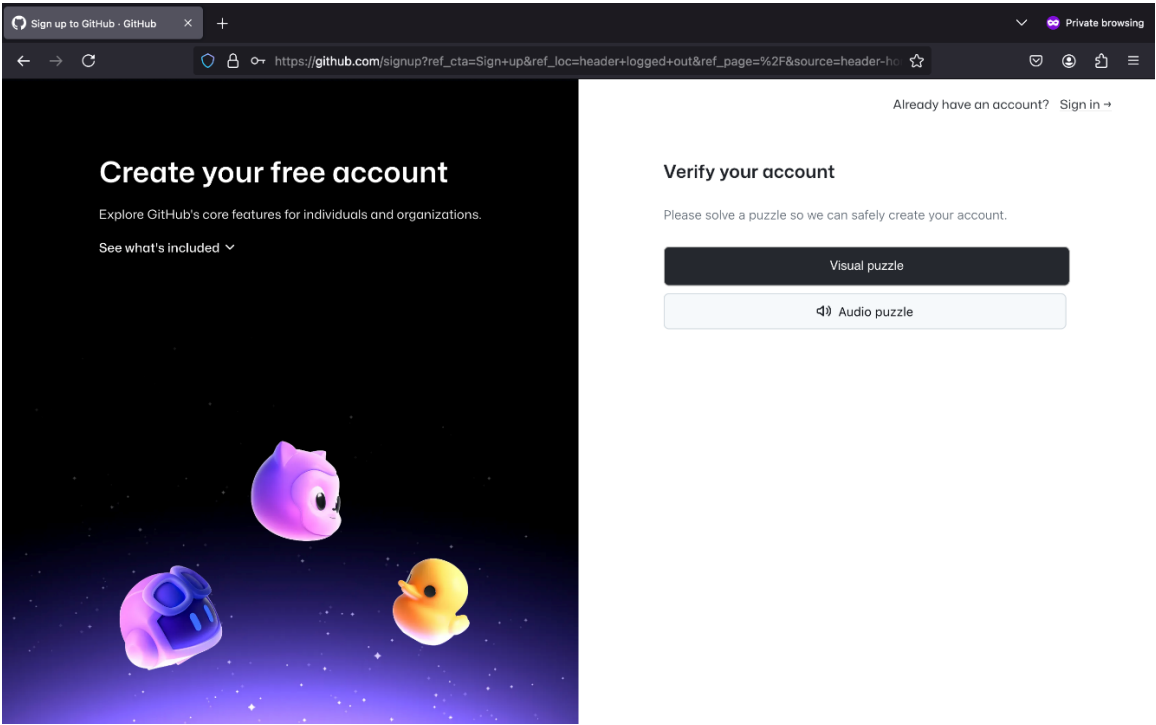


3. Enter your details in the **Email**, **Password**, and **Username** fields. Then, select **Continue**.

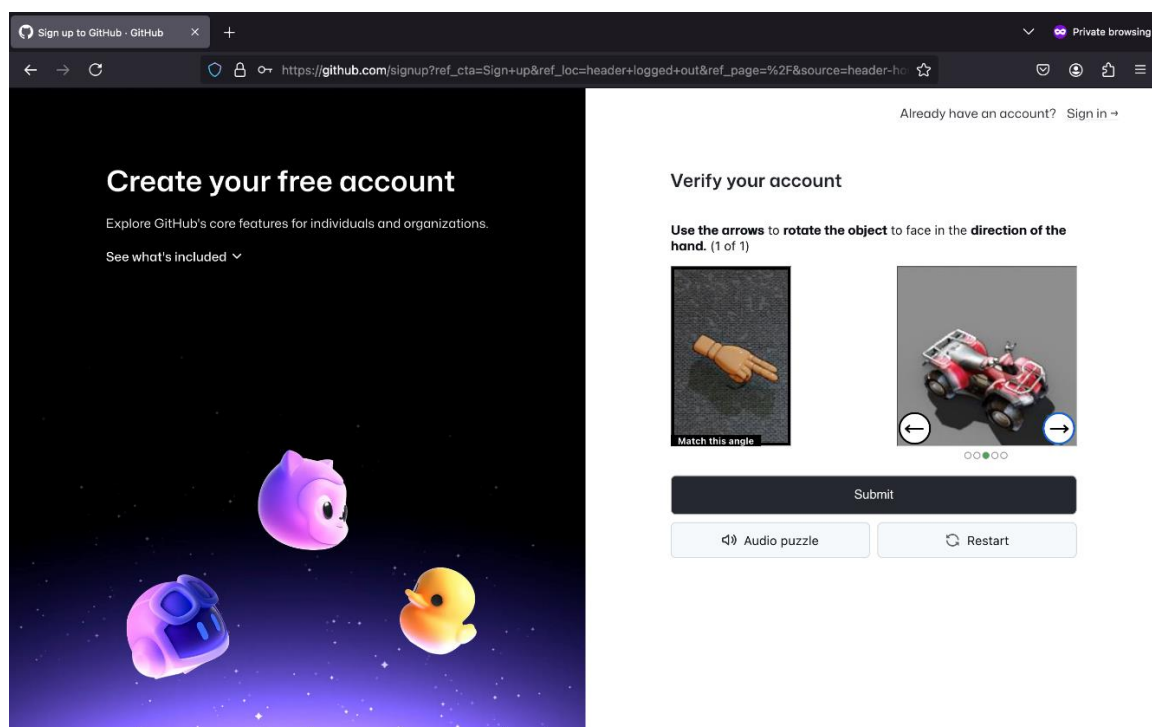
The following screenshot shows the “Sign up to GitHub” page.



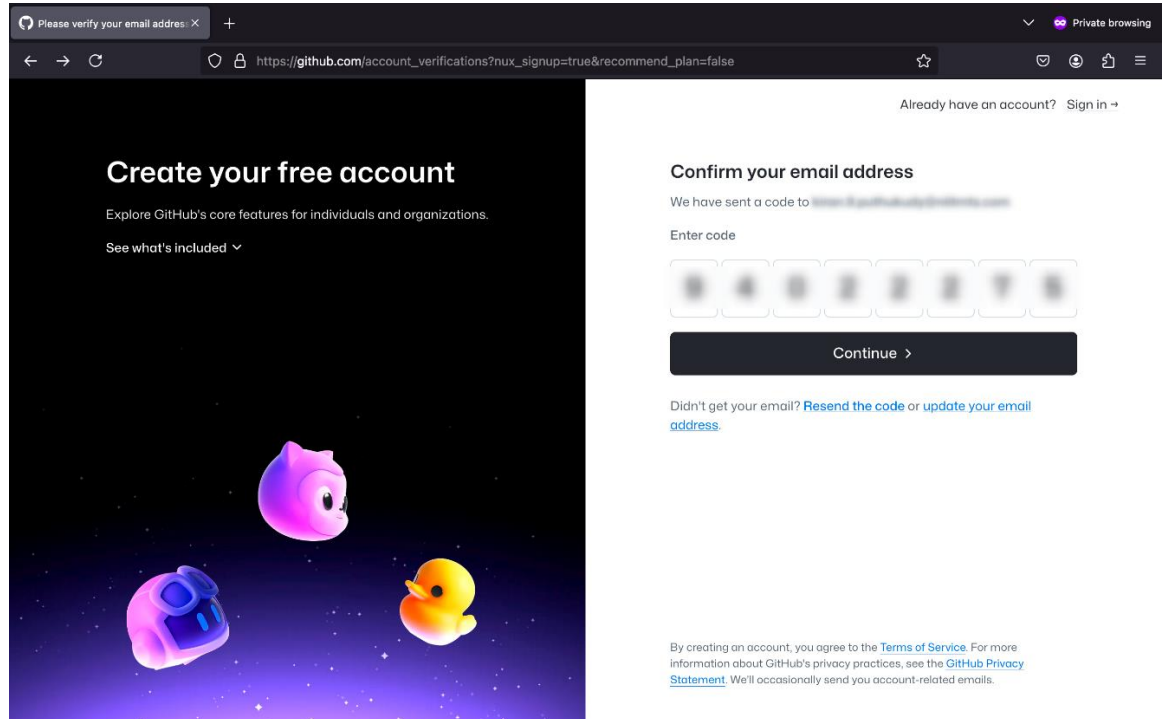
4. To verify your account, select the **Visual puzzle** button.



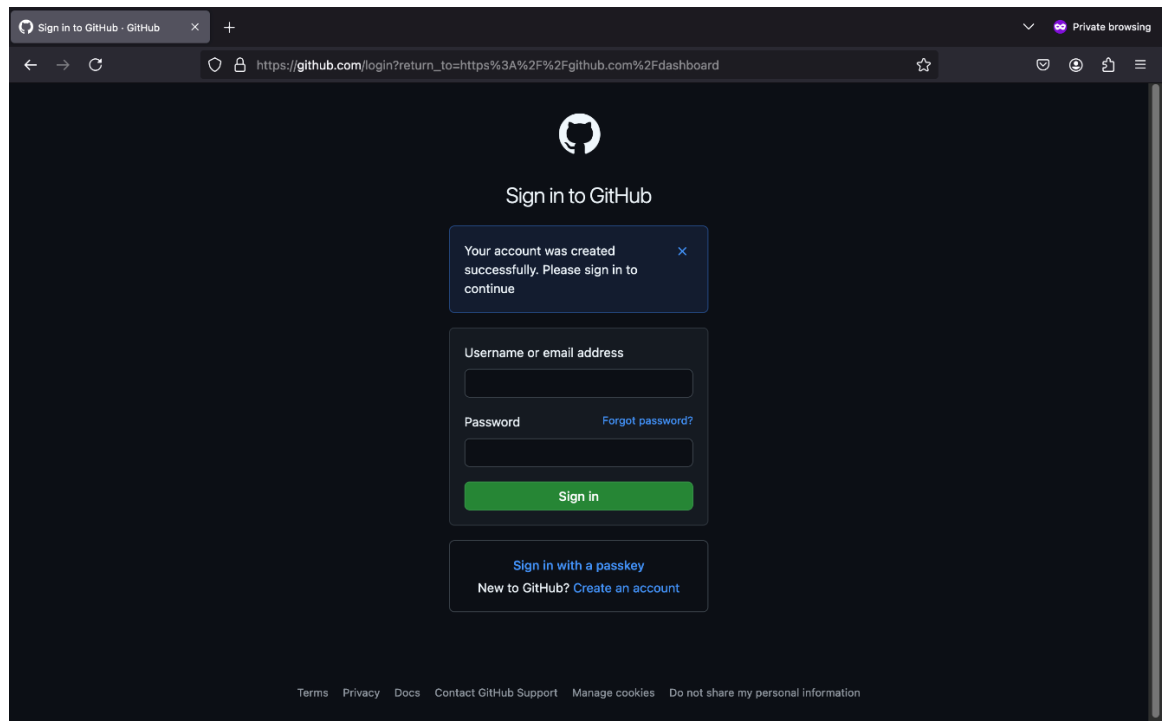
5. Solve the visual puzzle and select **Submit**.



6. To confirm your email, enter the confirmation code sent to your registered email in the **Enter code** field and select **Continue**.

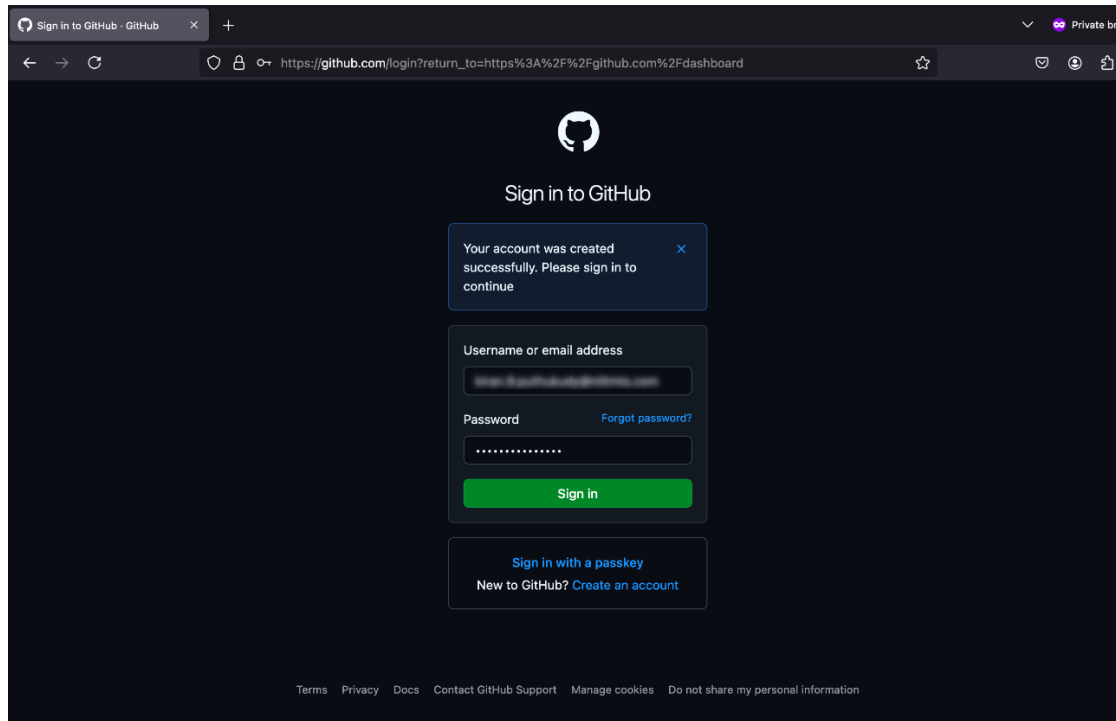


After your GitHub account has been successfully created, a confirmation message is displayed as shown in the following screenshot.

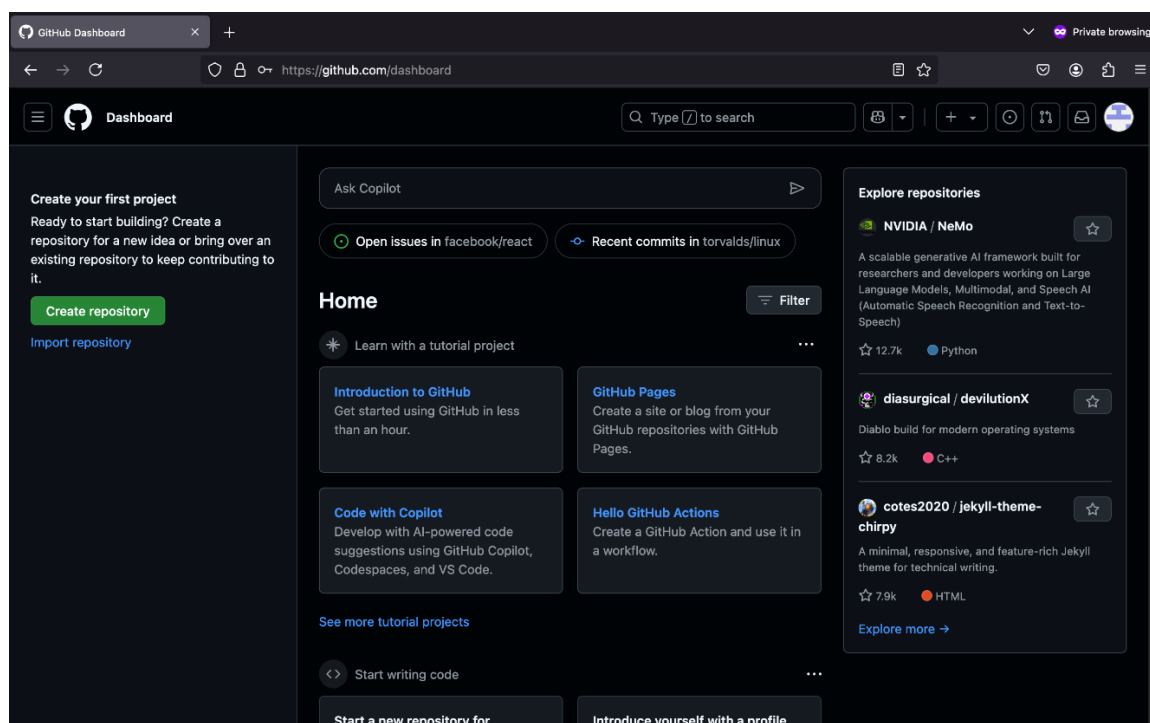


7. To sign in to your account, enter your credentials in the **Username or email address** and **Password** fields, and then select **Sign in**.

The following screenshot shows the GitHub sign-in page which includes fields for entering the username and password.



After you sign in, the GitHub dashboard appears as shown in the following screenshot.



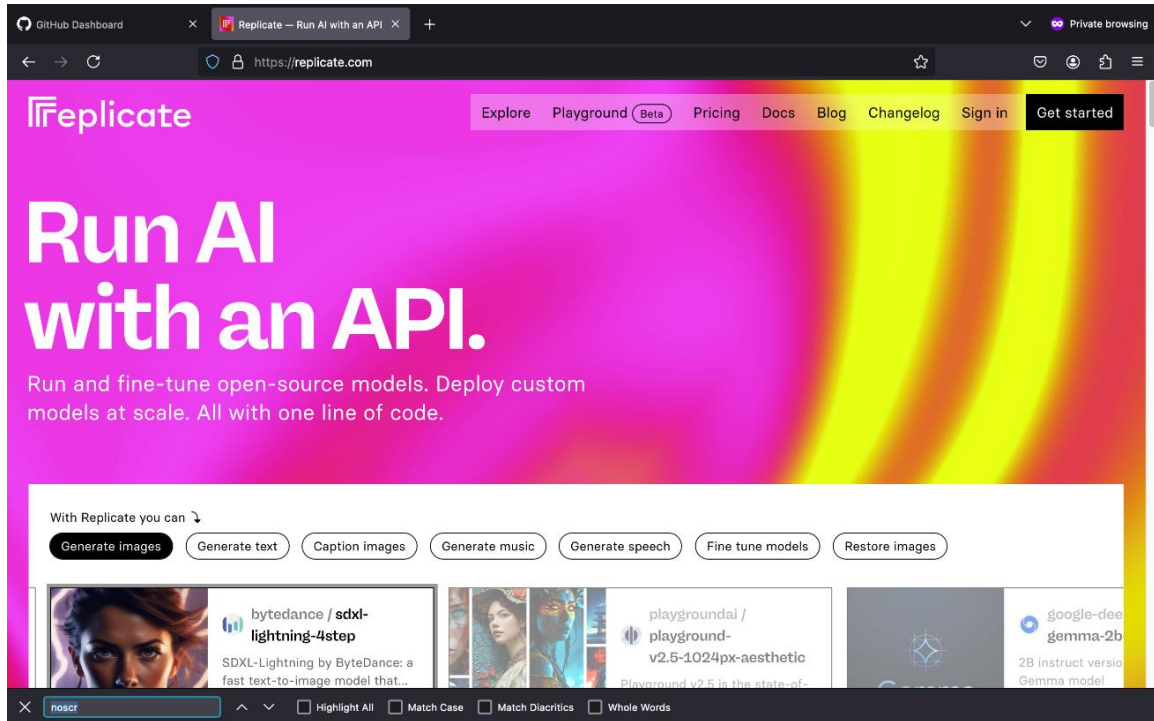
Create a Replicate account

Overview

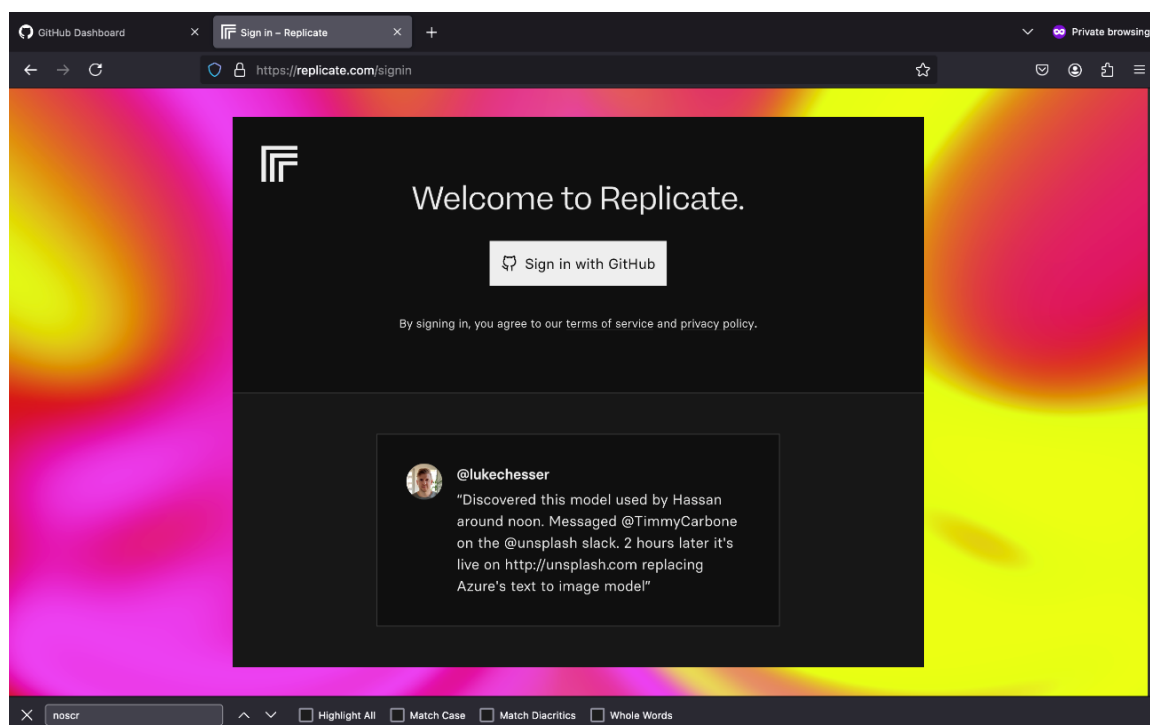
In this procedure, you'll use your GitHub account to register for a Replicate account. Replicate is a cloud-based platform that lets you use AI models such as IBM Granite without requiring advanced hardware. As part of this procedure, you'll create a Replicate token. A token is a secure access key that allows the lab environment to authenticate with Replicate and run models from your account in Google Colab.

Instructions

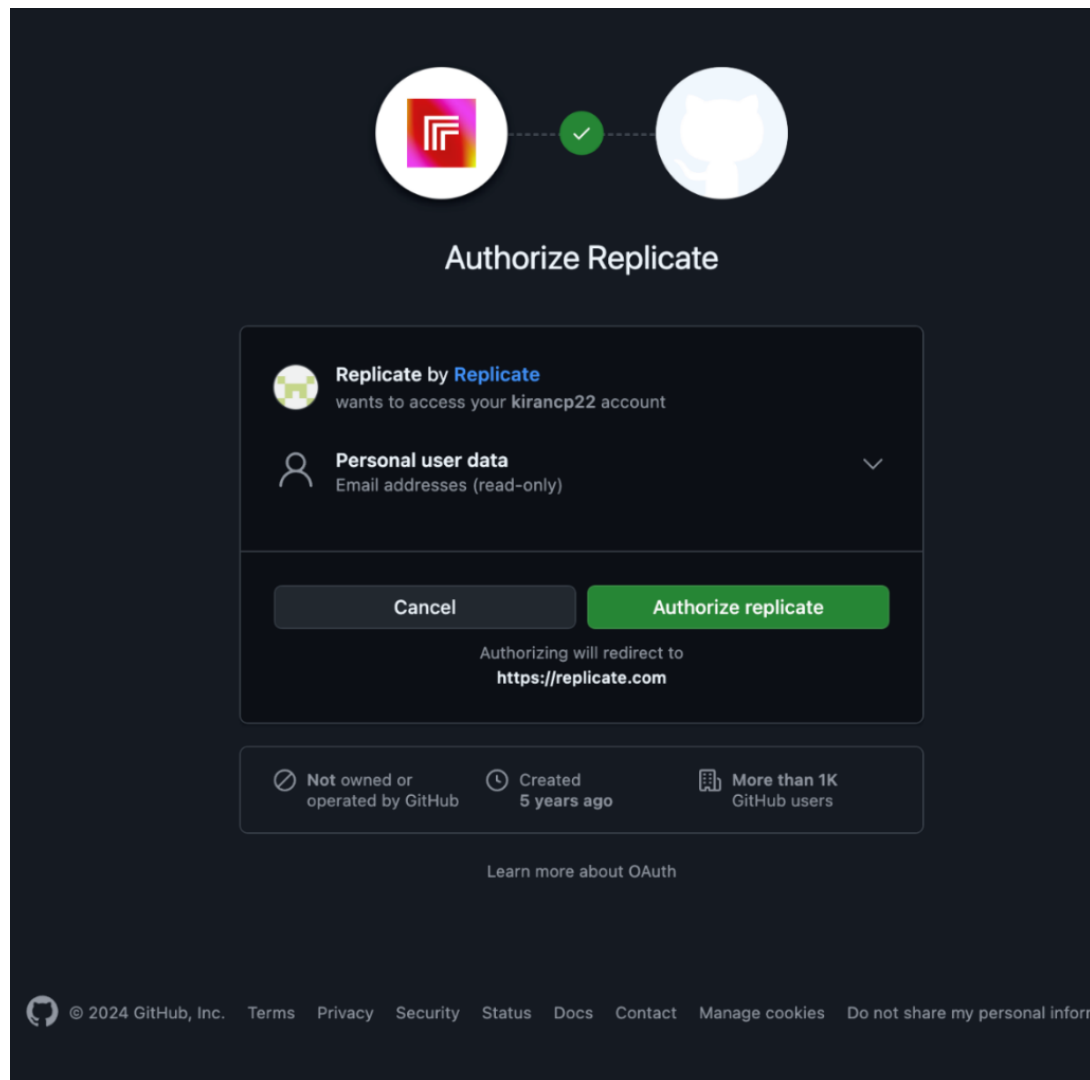
1. Go to the [Replicate](https://replicate.com) website.
2. Select **Get started**, as shown in the following screenshot.



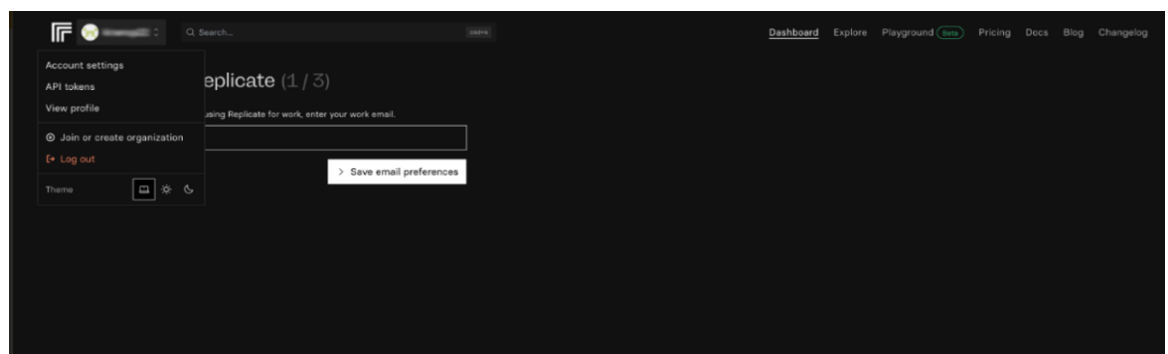
3. Select the **Sign in with GitHub** button, as shown in the following screenshot.



4. Select **Authorize replicate** to continue, as shown in the following screenshot.

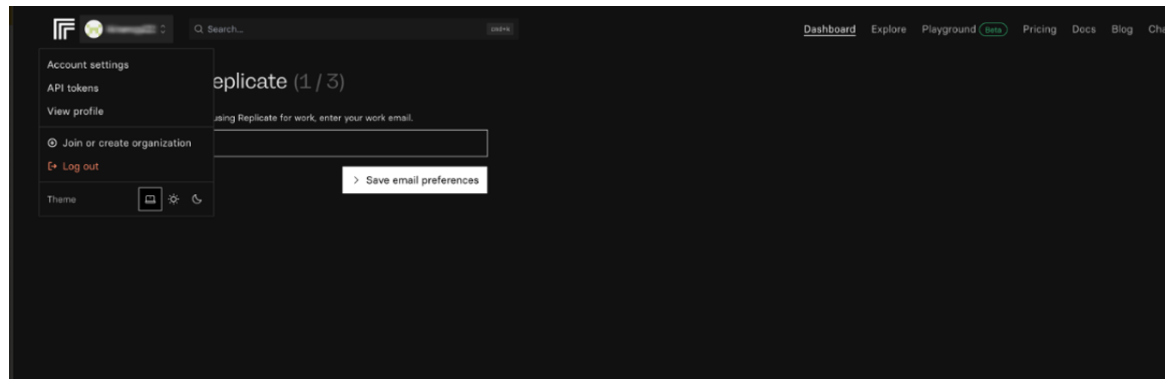


5. After your Replicate account is created, the Replicate dashboard appears as shown in the following screenshot. To create a Replicate token, select the **Account settings** option from the navigation menu.

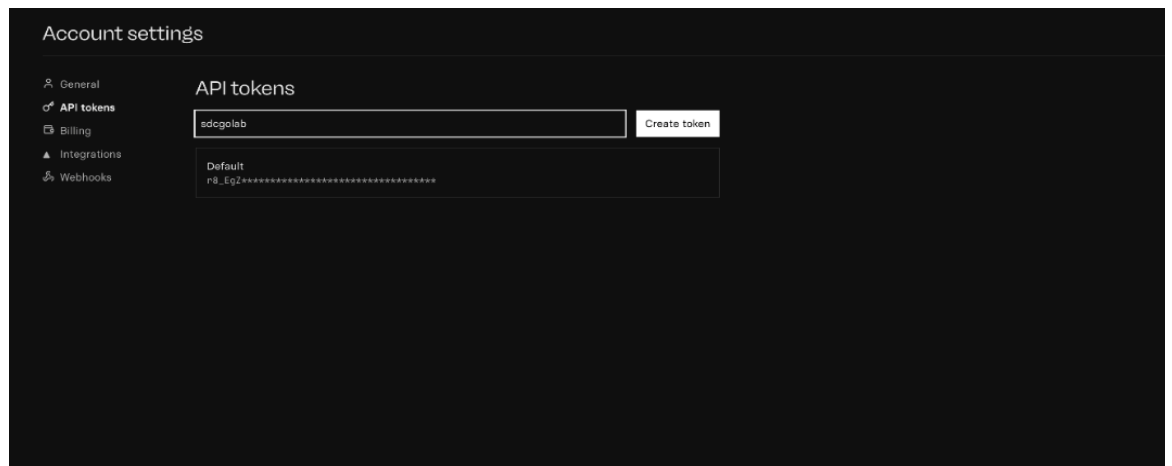


6. Select **API tokens** from the **Account settings** panel.

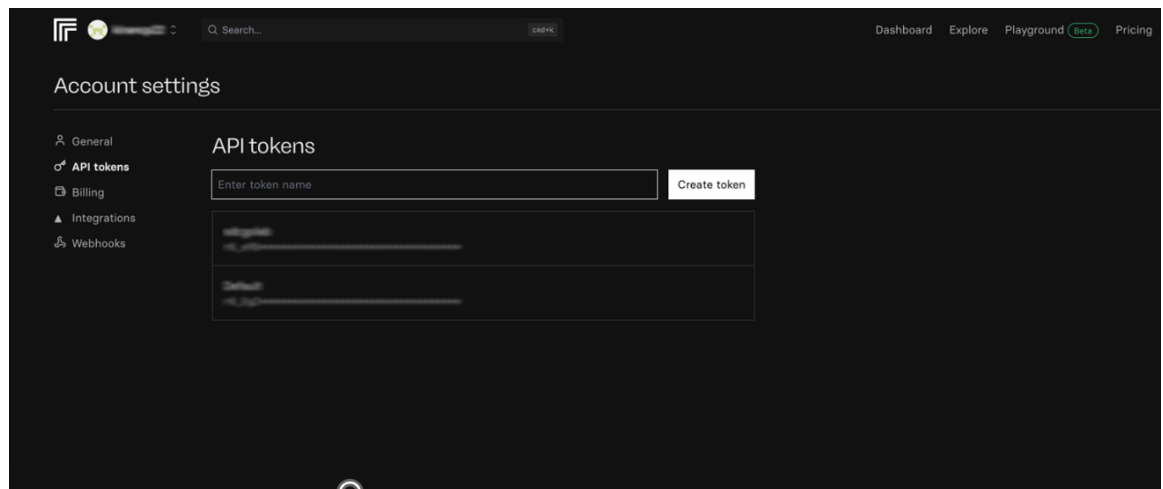
The following screenshot shows the Replicate dashboard.



7. Enter a name for the token in the **API tokens** field and then select **Create token**, as shown in the following screenshot.

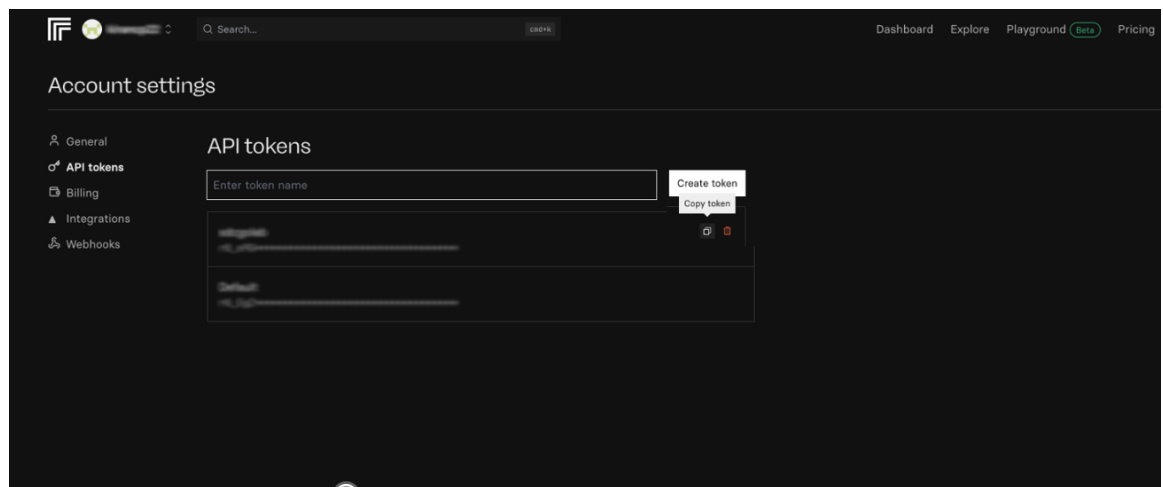


After your API token is created, it should be displayed on the “Account settings” page as shown in the following screenshot.

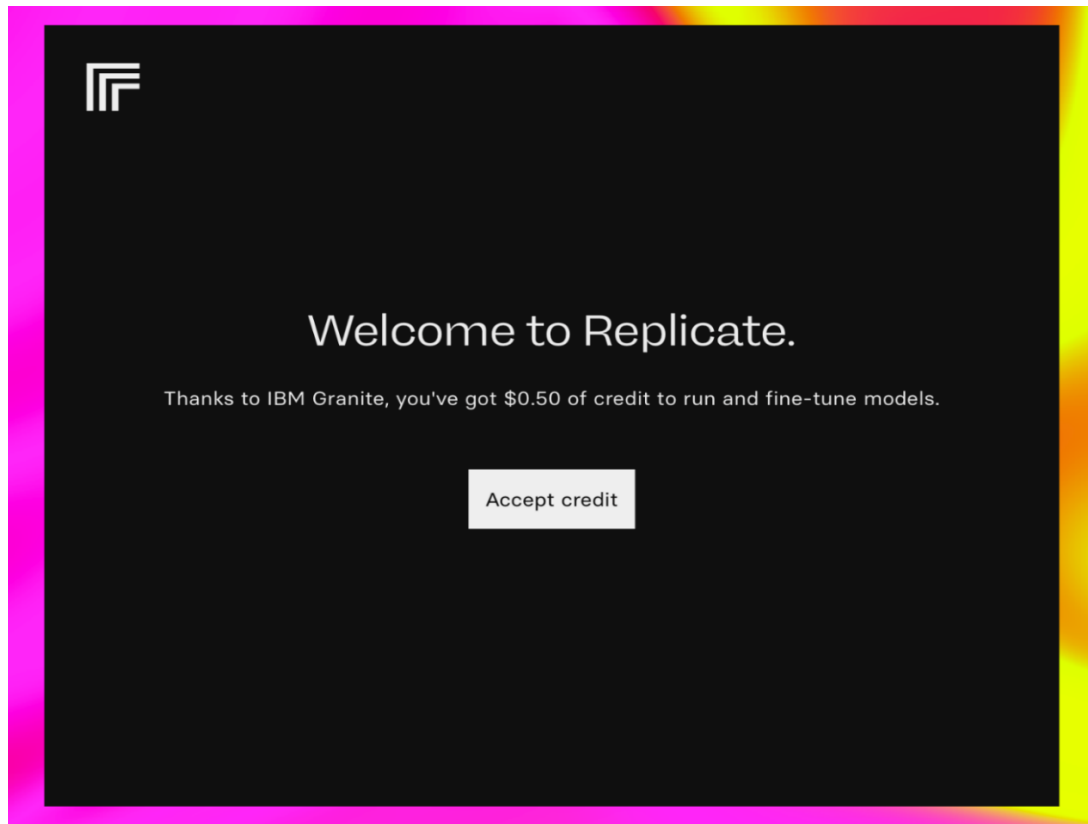


8. To copy the Replicate API, select the **Copy token** icon, as shown in the following screenshot.

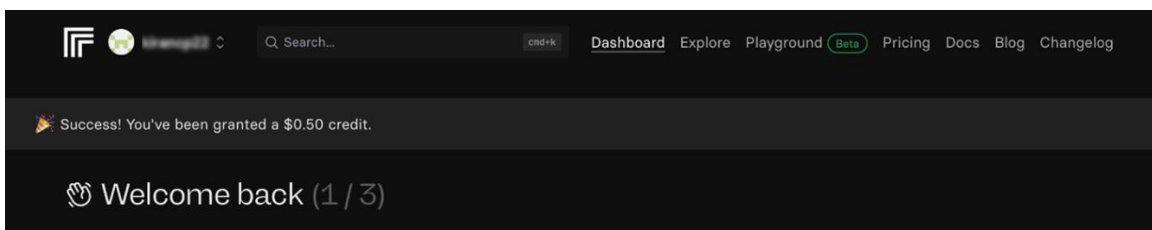
Note: Save the Replicate token because you'll need it to authenticate with the Replicate API when running code in the Google Colab environment later in this lab.



9. To run the lab without interruptions, you'll require some Replicate credits. Go to the [Replicate invite](#) link to claim your free \$0.50 credit.
10. To claim the amount, select **Accept credit**.



A confirmation message is displayed indicating that a \$0.50 credit has been added to your Replicate account as shown in the following screenshot.



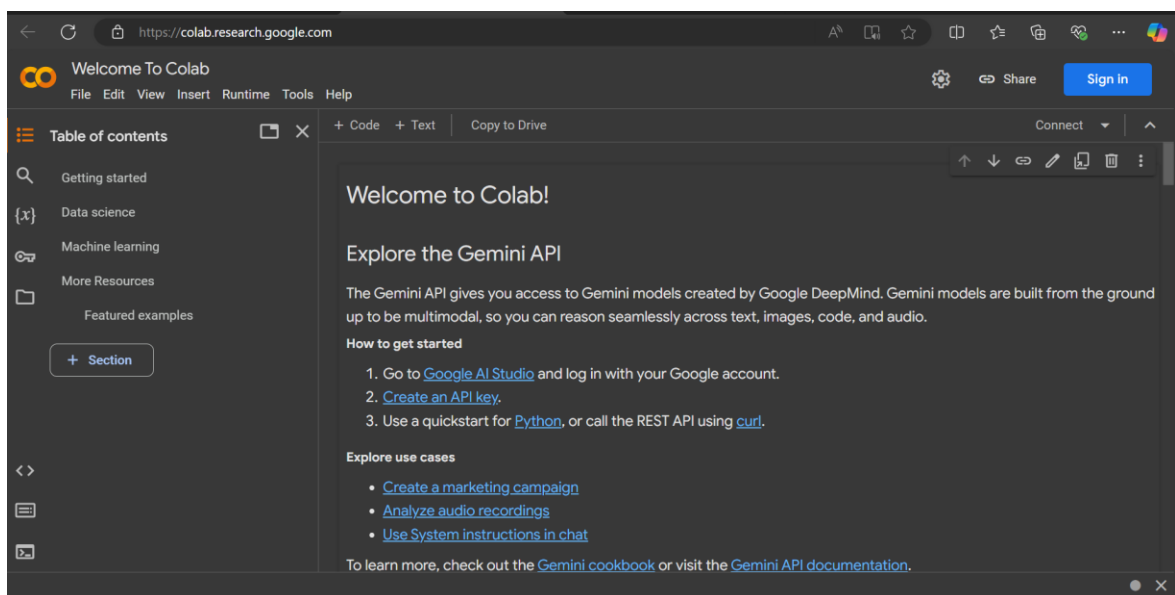
Sign up for Google Colab

Overview

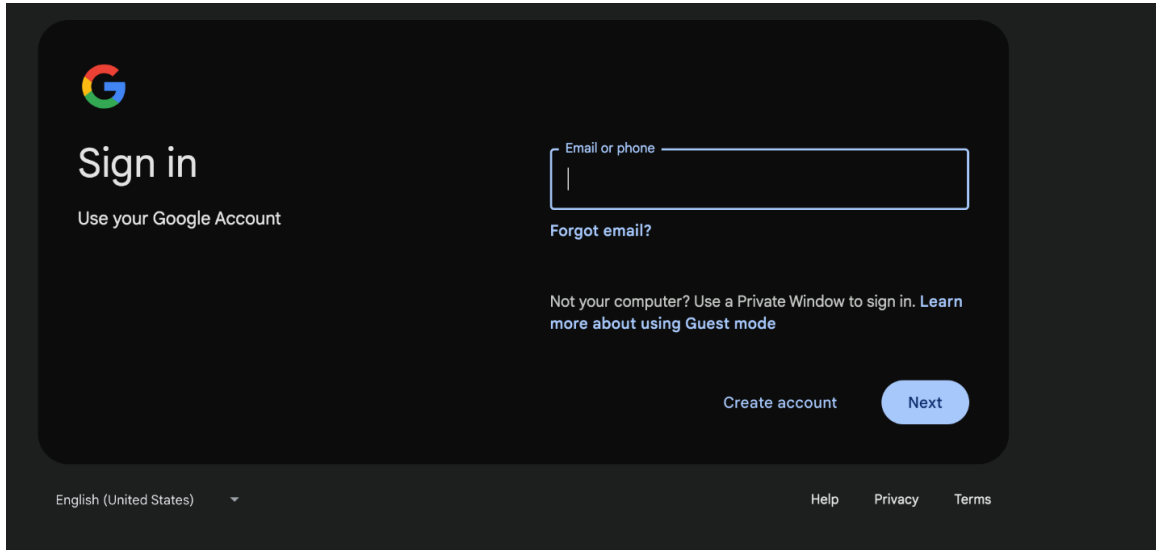
In this procedure, you'll set up a Google Colab account. Google Colab is a free cloud platform that lets you run code in notebooks, which are commonly used for machine learning, data science, and AI tasks. A Google Colab account allows you to install and use the tools needed to complete this lab.

Instructions

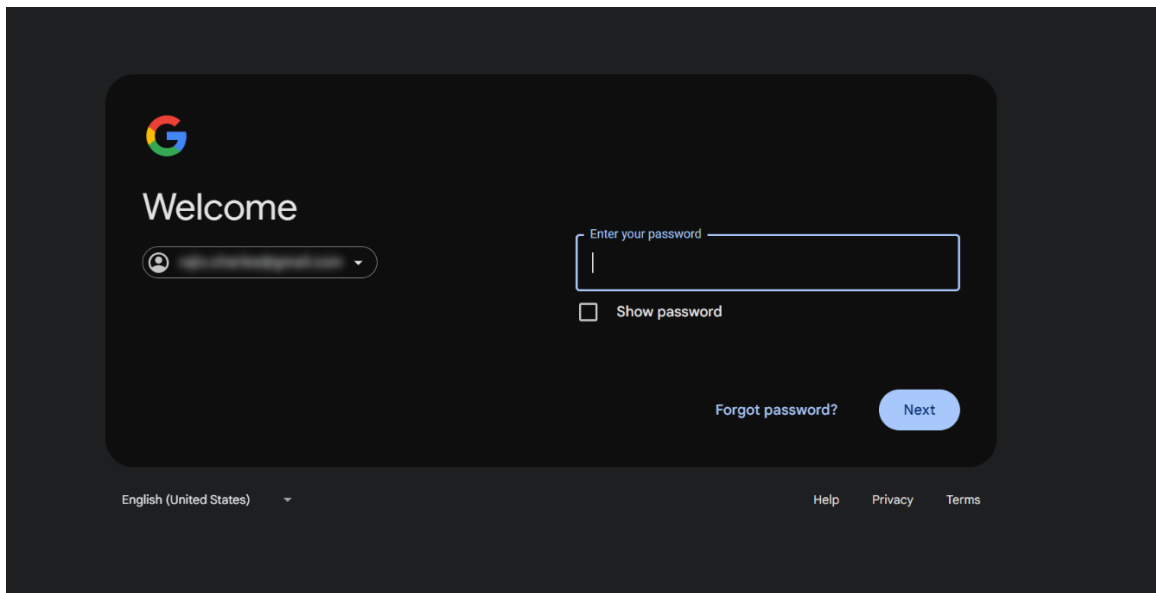
1. To sign up, go to the [Google Colab](https://colab.research.google.com) website.
2. Select **Sign in**, as shown in the following screenshot.



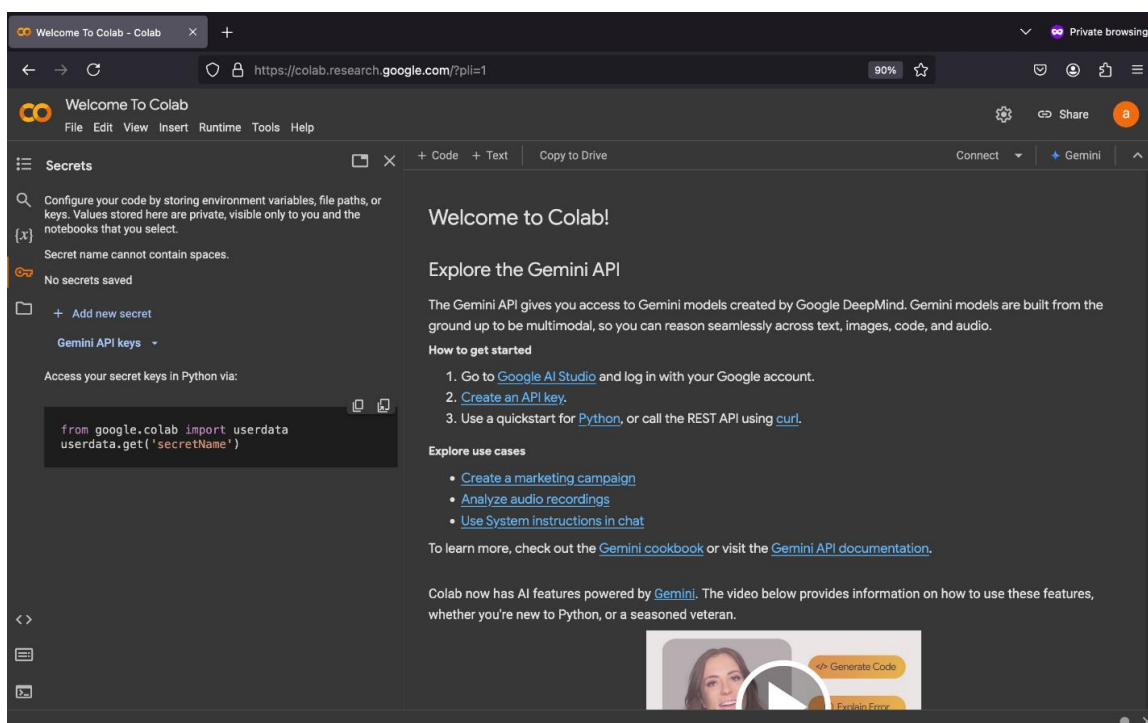
3. Enter your email or phone number in the **Email or phone** field, and then select **Next**. The following screenshot shows the Google sign-in page.



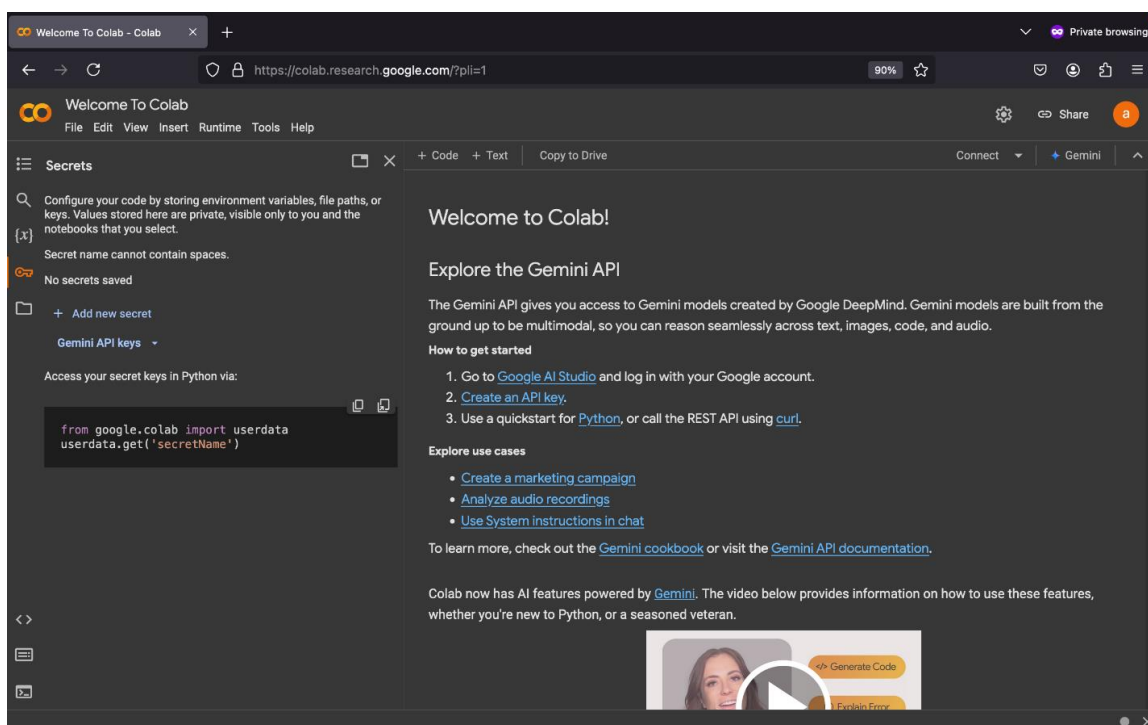
4. Enter your password in the **Enter your password** field, and then select **Next**.



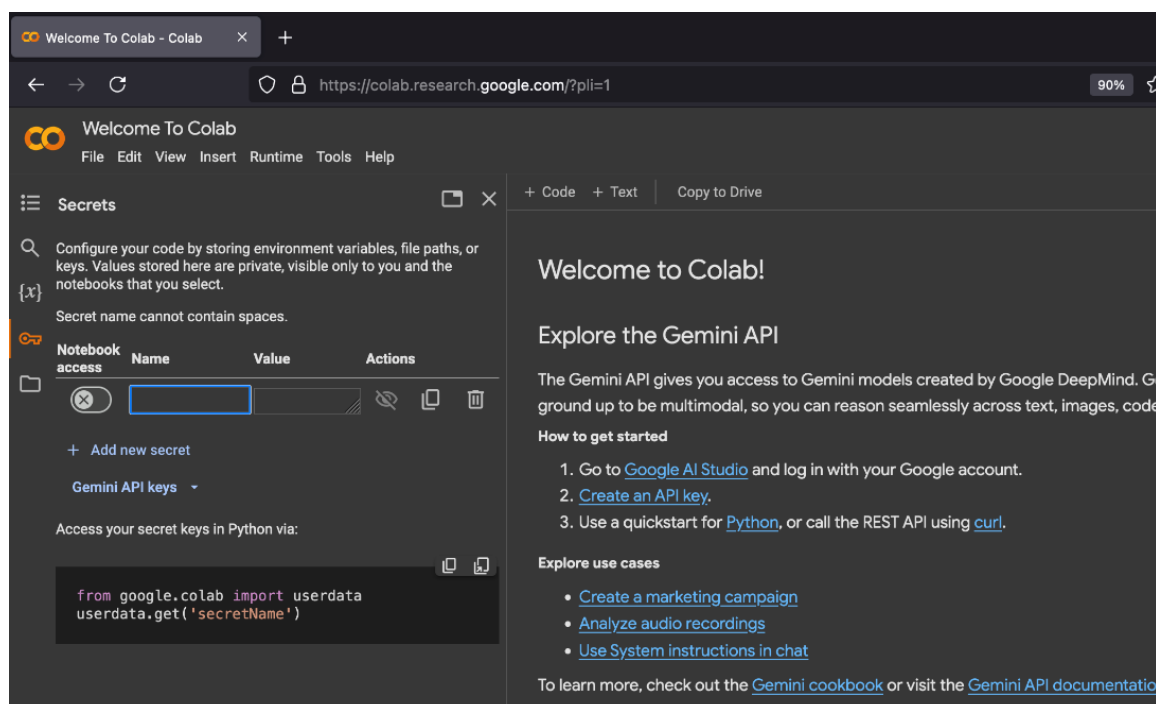
5. To store your Replicate API token, select the key icon from the **Secrets** tab on the Welcome to Colab page, as shown in the following screenshot.



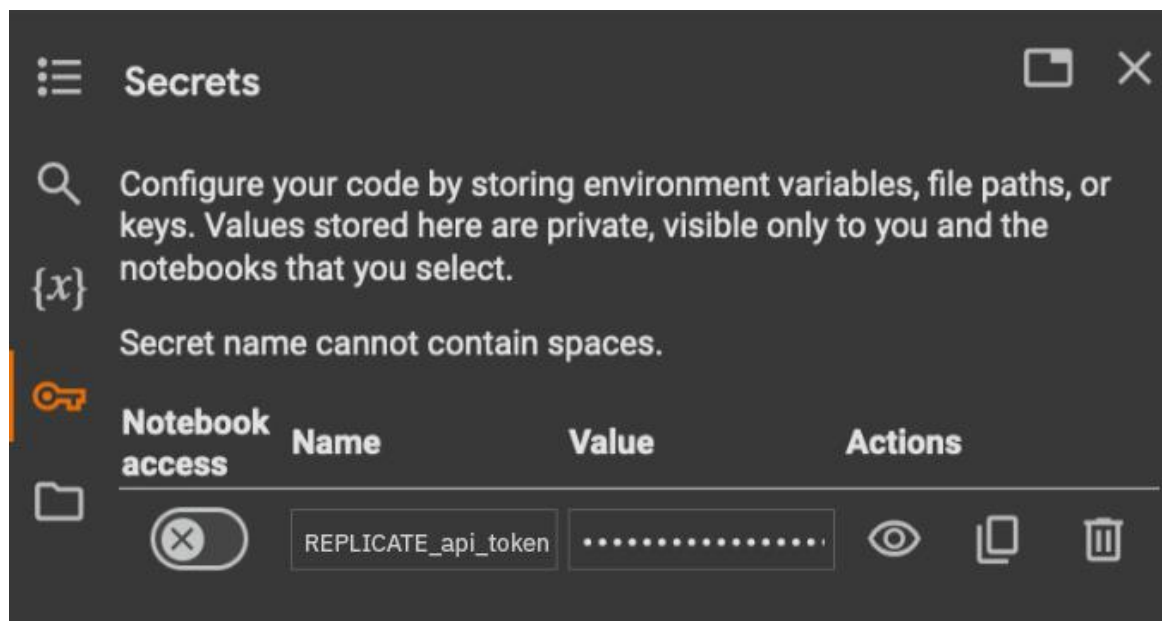
6. Select **Add new secret**.



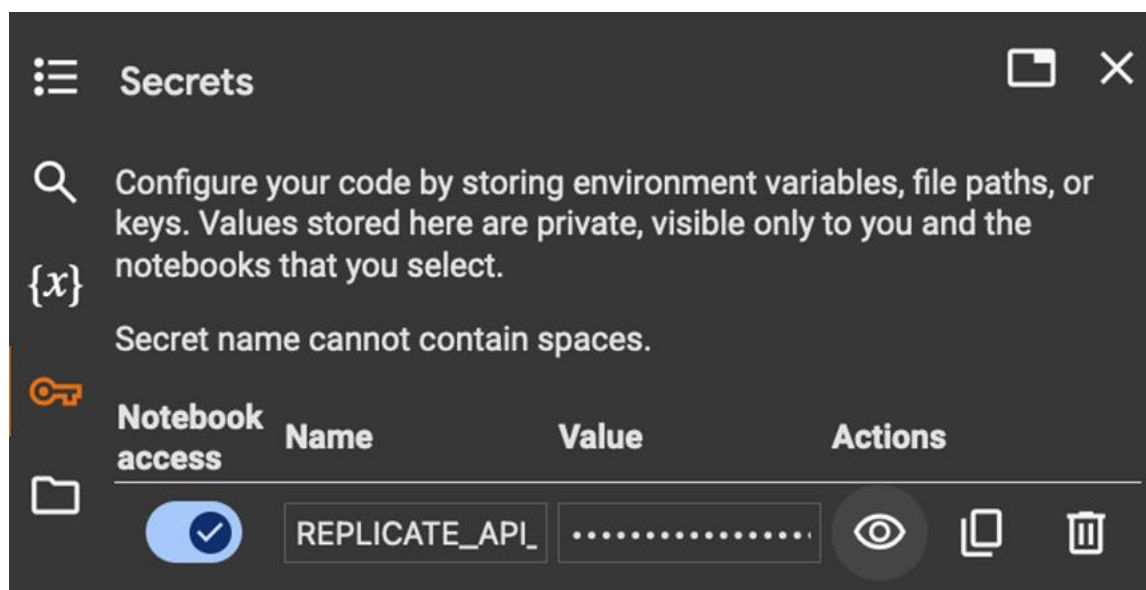
7. Type REPLICATE_api_token in the **Name** field.



8. Paste your Replicate API token you copied into the **Value** field, as shown in the following screenshot.



9. Set the **Notebook access** switch on, as shown in the following screenshot. Then, select the close icon to exit the configuration.



Initialize the AI agent and tools

Overview

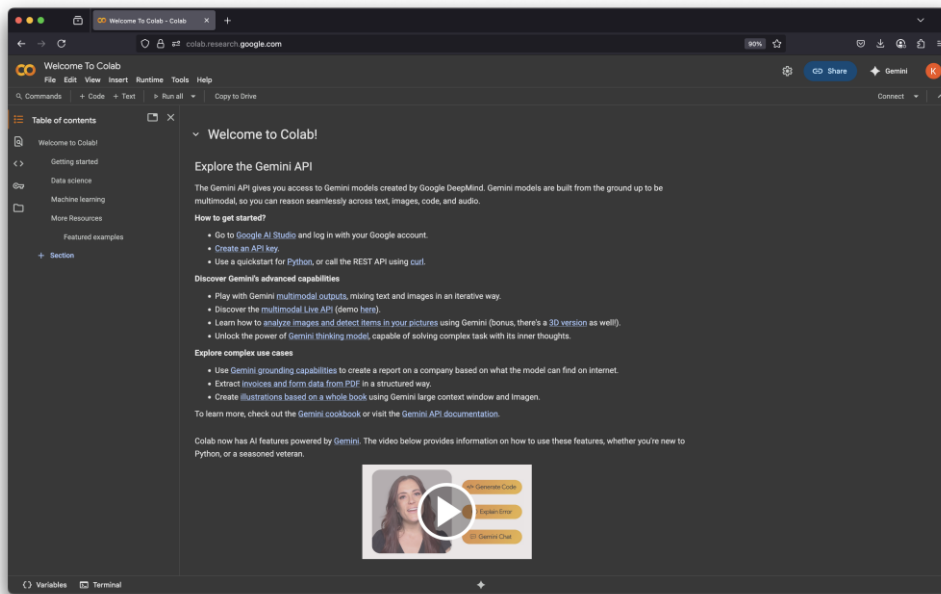
In this procedure you'll initialize TechMart's AI agent and the tools it uses by loading a Jupyter notebook.

In the context of an AI agent, **tools** are functions that help an AI agent perform specific tasks such as searching for information, checking product stock, or looking up data. The user provides prompts that define which tools the agent can access by including them in the agent's setup. When responding, the agent sends a query to the selected tool, receives the result, and uses that information to generate its reply.

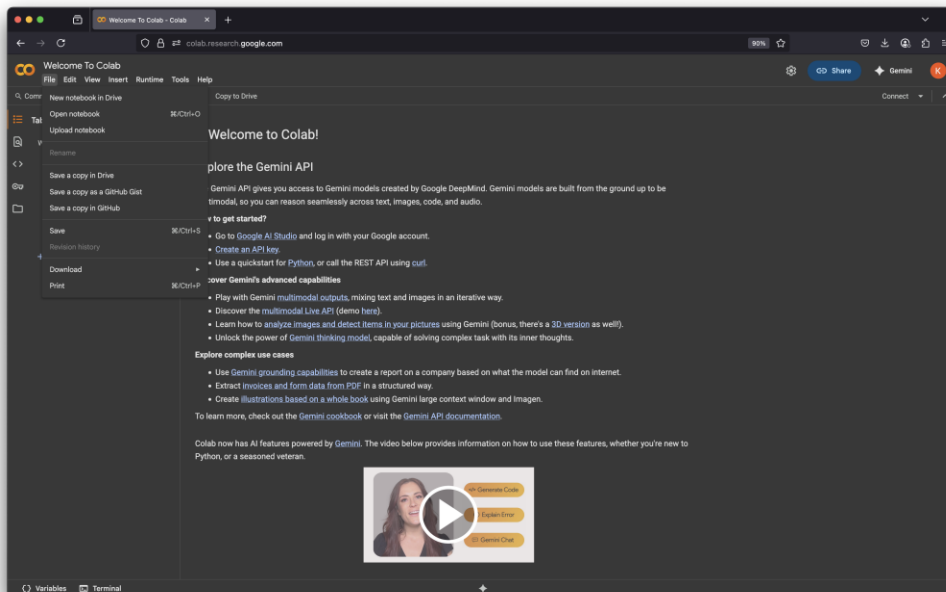
A **Jupyter notebook** is a shareable document that combines computer code, plain language descriptions, data, and visualizations.

Instructions

1. Select [Google Colab](#) to open Google Colab as shown in the following screenshot. To begin working on the notebook, select **File** from the menu.

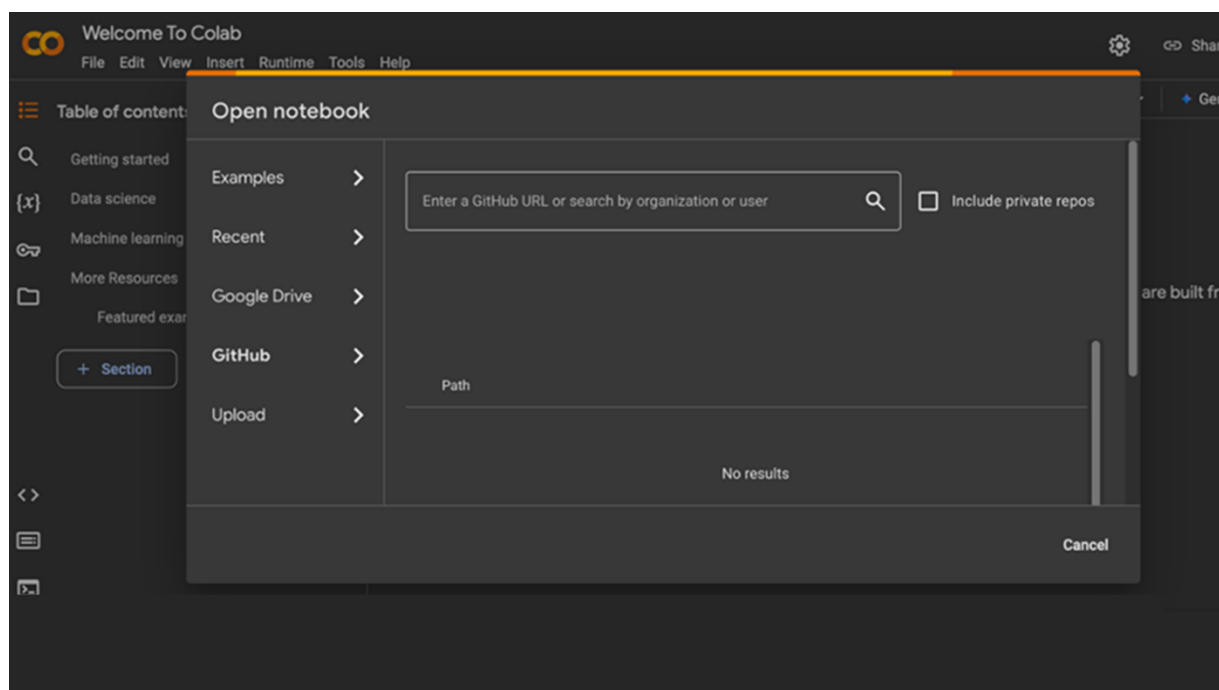


2. Select **Open notebook**.

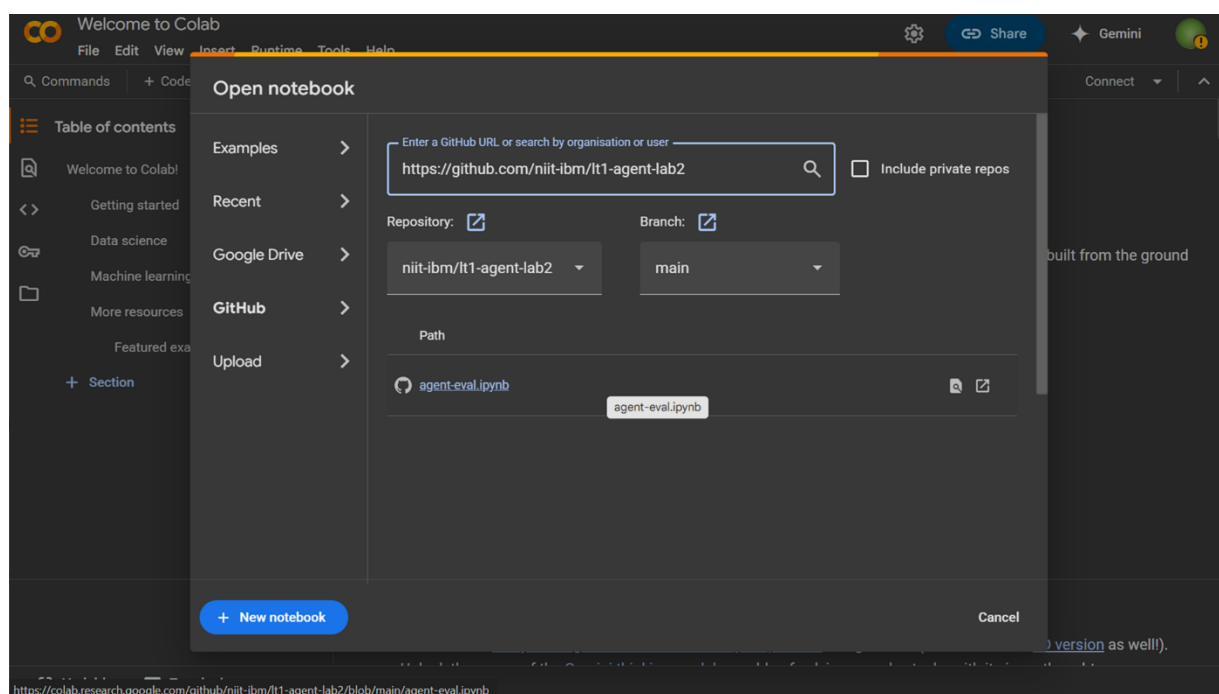


3. In the “Open notebook” dialog, select **GitHub**.

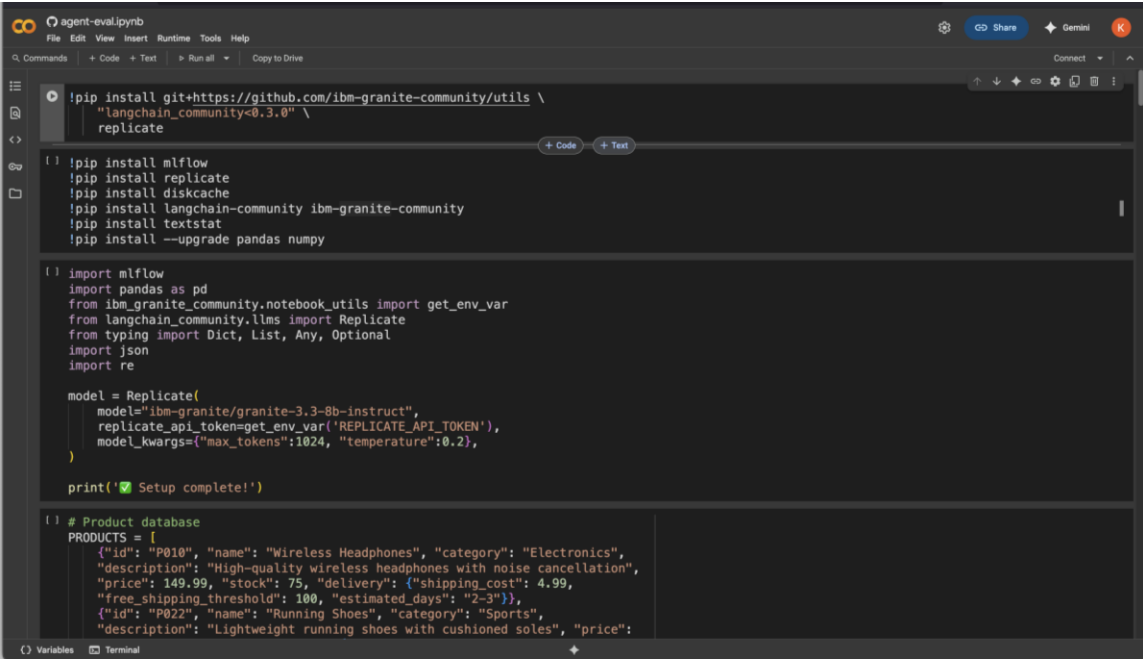
To find and load the Jupyter notebook directly in Google Colab, type the uniform resource locator (URL) <https://github.com/niit-ibm/lt1-agent-lab2> in the **Enter a GitHub URL or search by organization or user** field. Then, press Enter.



4. After you enter the URL, the **Repository** and **Branch** fields are automatically populated. To open the notebook, select **agent-eval.ipynb**.



The agent-eval.ipynb notebook opens in the Colab workspace, as shown in the following screenshot.



```
!pip install git+https://github.com/ibm-granite-community/utils \
    "langchain_community<0.3.0" \
    replicate

!pip install mlflow
!pip install replicate
!pip install diskcache
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy

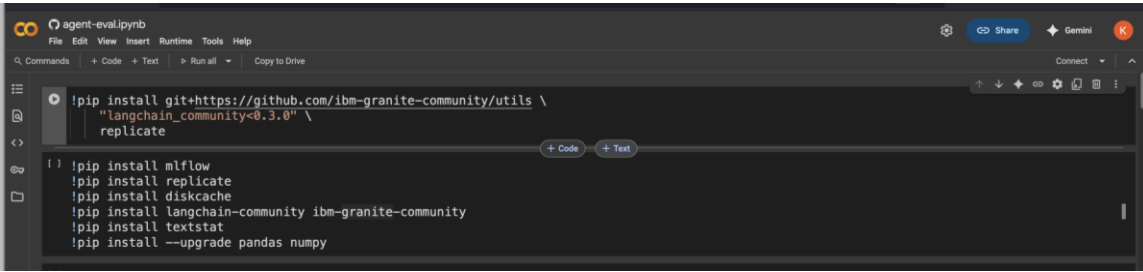
import mlflow
import pandas as pd
from ibm_granite_community.notebook_utils import get_env_var
from langchain_community.llms import Replicate
from typing import Dict, List, Any, Optional
import json
import re

model = Replicate(
    model="ibm-granite/granite-3.3-8b-instruct",
    replicate_api_token=get_env_var('REPLICATE_API_TOKEN'),
    model_kwargs={"max_tokens":1024, "temperature":0.2},
)

print('✅ Setup complete!')

# Product database
PRODUCTS = [
    {"id": "P010", "name": "Wireless Headphones", "category": "Electronics",
     "description": "High-quality wireless headphones with noise cancellation",
     "price": 149.99, "stock": 75, "delivery": {"shipping_cost": 4.99,
     "free_shipping_threshold": 100, "estimated_days": "2-3"}},
    {"id": "P022", "name": "Running Shoes", "category": "Sports",
     "description": "Lightweight running shoes with cushioned soles", "price":
```

5. To run the code, select the **Connect** drop-down arrow.



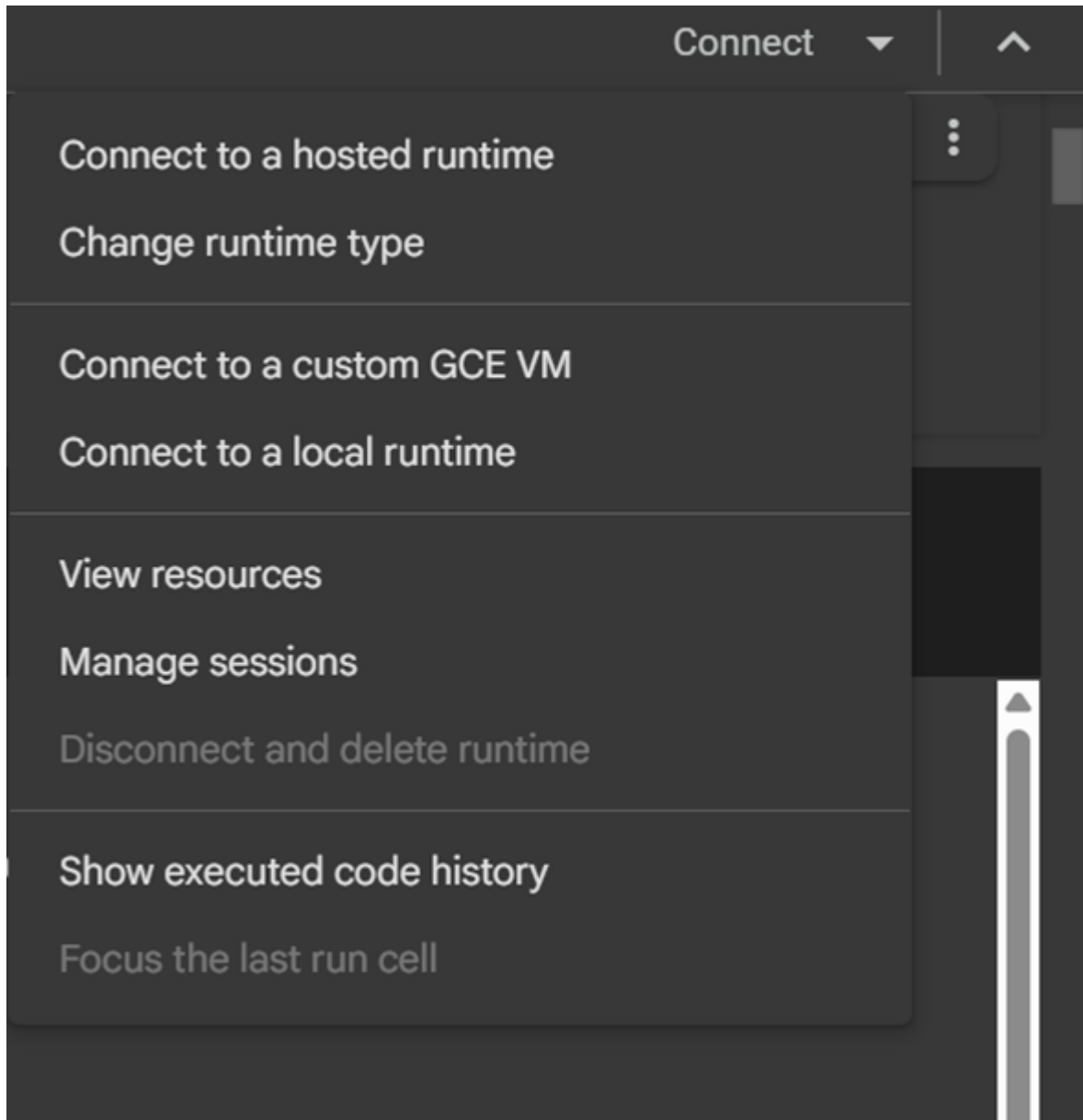
```
!pip install git+https://github.com/ibm-granite-community/utils \
    "langchain_community<0.3.0" \
    replicate

!pip install mlflow
!pip install replicate
!pip install diskcache
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy
```

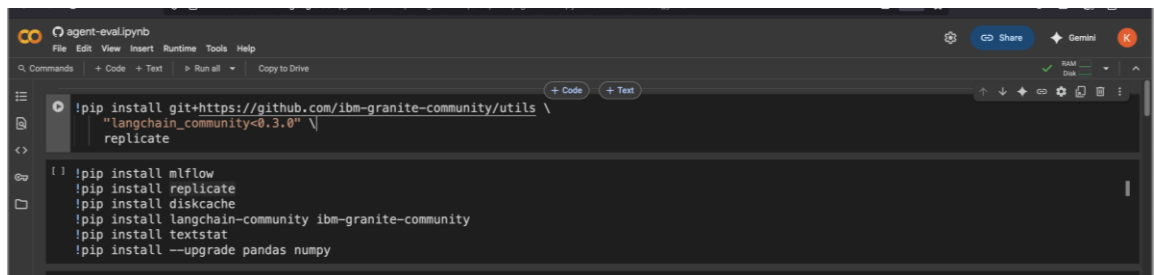
6. From the list, select **Connect to a hosted runtime**.

The following screenshot shows a Jupyter notebook with the Connect to a hosted runtime option in the Connect list.

Note: When you use Colab, your code doesn't run on your local machine. Instead, it runs in a cloud-based environment provided by Google, known as a **hosted runtime**. This means you don't need to install Python or any libraries on your device; they're already available in the cloud environment.

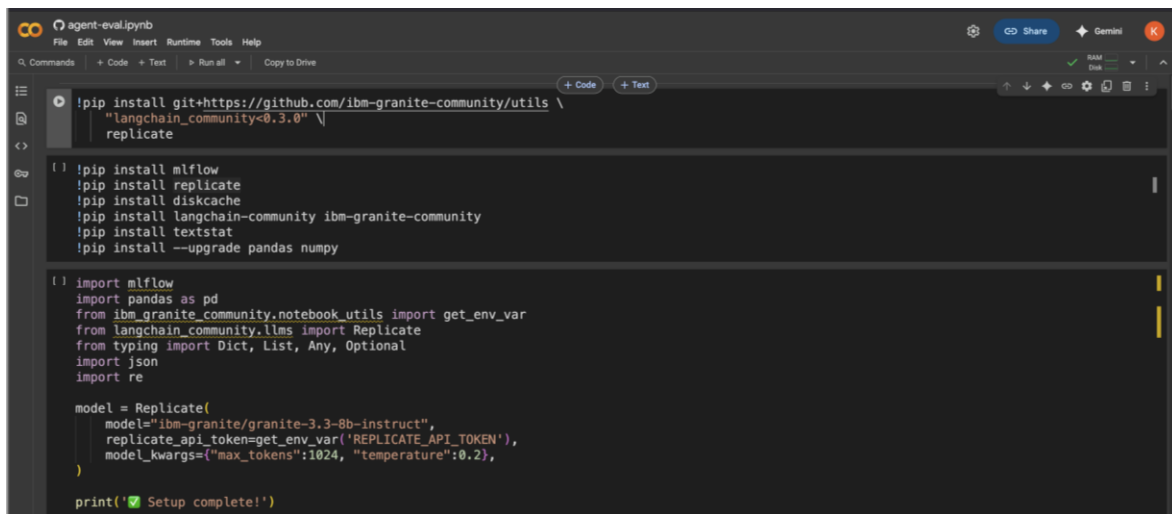


Check the **Colab** toolbar for a green checkmark, as shown in the following screenshot. This indicates that the connection is successful, and your code is ready to run.



7. To enable your AI agent to work with the IBM Granite model and generate helpful responses, certain libraries must be installed. In the first cell, select the **Run** icon to begin the installation of the libraries.

Note: In a Jupyter notebook, each row is called a **cell**. These cells are not numbered but are organized to run sequentially, beginning from the first cell. Each code cell includes a **Run** icon (▶), which you use to execute the code within that cell.



```
!pip install git+https://github.com/ibm-granite-community/utls \
"langchain_community<0.3.0" \
replicate

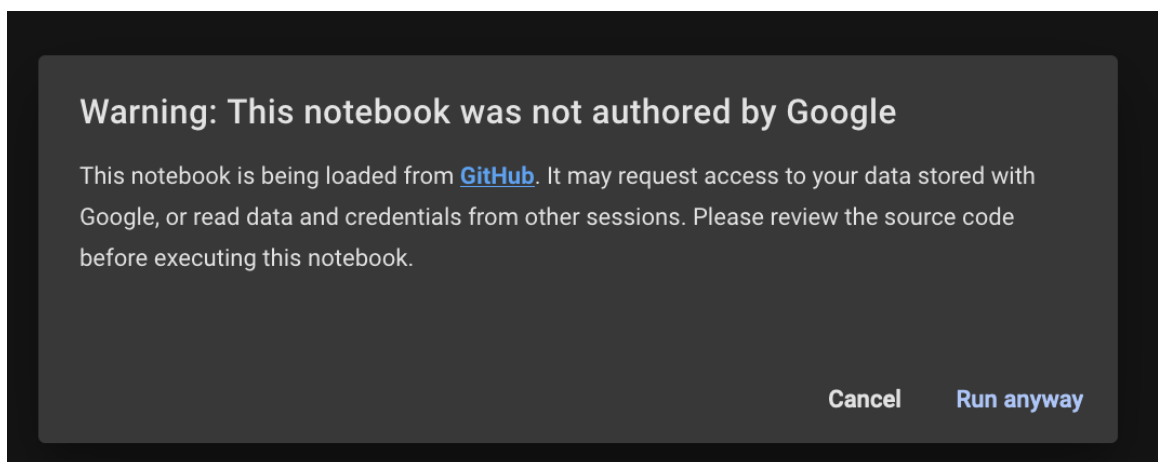
!pip install mlflow
!pip install replicate
!pip install diskcache
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy

import mlflow
import pandas as pd
from ibm_granite_community.notebook_utils import get_env_var
from langchain_community.llms import Replicate
from typing import Dict, List, Any, Optional
import json
import re

model = Replicate(
    model="ibm-granite/granite-3.3-8b-instruct",
    replicate_api_token=get_env_var('REPLICATE_API_TOKEN'),
    model_kwargs={"max_tokens":1024, "temperature":0.2},
)

print('✅ Setup complete!')
```

8. Select **Run anyway** to continue loading the required libraries.



A green checkmark is displayed next to the **Run** icon, indicating that the required libraries have been successfully installed.

```

!pip install git+https://github.com/ibm-granite-community/utlis \
"langchain_community<0.3.0" \
replicate

Requirement already satisfied: typing_extensions==4.5.0 in /usr/local/lib/python3.11/dist-packages (from replicate) (4.14.0)
Requirement already satisfied: aiohttp==4.0.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0==>3.8.3->langchain_community<0.3.0) (2.6.1)
Requirement already satisfied: aiohttp==4.0.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0==>3.8.3->langchain_community<0.3.0) (1.3.2)
Requirement already satisfied: attrs==17.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0==>3.8.3->langchain_community<0.3.0) (25.3.0)
Requirement already satisfied: frozenlist==1.1.1 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0==>3.8.3->langchain_community<0.3.0) (1.6.0)
Requirement already satisfied: multidict==7.0.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0==>3.8.3->langchain_community<0.3.0) (6.4.4)
Requirement already satisfied: propcache==0.2.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0==>3.8.3->langchain_community<0.3.0) (0.3.1)
Requirement already satisfied: yarl==2.0.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0==>3.8.3->langchain_community<0.3.0) (1.20.0)
Collecting marshmallow==4.0.0 in /usr/local/lib/python3.11/dist-packages (from dataclasses-json==0.7.0==>0.5.7->langchain_community<0.3.0)
  Downloading marshmallow-3.26.1-py3-none-any.whl.metadata (7.3 kB)
Collecting typing_inspect==0.9.0 in /usr/local/lib/python3.11/dist-packages (from dataclasses-json==0.7.0==>0.5.7->langchain_community<0.3.0)
  Downloading typing_inspect-0.9.0-py3-none-any.whl.metadata (1.5 kB)
Requirement already satisfied: aiohttp==4.0.0 in /usr/local/lib/python3.11/dist-packages (from httpx==0.21.0->replicate) (4.9.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.11/dist-packages (from httpx==0.21.0->replicate) (2025.4.26)
Requirement already satisfied: httpcore==1.0.0 in /usr/local/lib/python3.11/dist-packages (from httpx==0.21.0->replicate) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.11/dist-packages (from httpx==0.21.0->replicate) (3.10)
Requirement already satisfied: h11==0.16 in /usr/local/lib/python3.11/dist-packages (from httpcore==1.0.0->httpx==0.21.0->replicate) (0.16.0)
Collecting langchain-text-splitters==0.3.0 in /usr/local/lib/python3.11/dist-packages (from langchain==0.3.0==>0.2.17->langchain_community<0.3.0)
  Downloading langchain_text_splitters-0.2.4-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: jsonpatch==2.0.0 in /usr/local/lib/python3.11/dist-packages (from langchain-core==0.3.0==>0.2.43->langchain_community<0.3.0) (1.33)
Requirement already satisfied: orjson==4.0.0 in /usr/local/lib/python3.11/dist-packages (from langchain-core==0.3.0==>0.2.43->langchain_community<0.3.0) (3.10.18)
Requirement already satisfied: requests-toolbelt==2.0.0 in /usr/local/lib/python3.11/dist-packages (from langsmith==0.2.0==>0.1.112->langchain_community<0.3.0) (1.0.0)
Requirement already satisfied: annotated-types==0.6.0 in /usr/local/lib/python3.11/dist-packages (from pydantic==1.10.7->replicate) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.11/dist-packages (from pydantic==1.10.7->replicate) (2.33.2)
Requirement already satisfied: typing-inspection==0.4.0 in /usr/local/lib/python3.11/dist-packages (from pydantic==1.10.7->replicate) (0.4.1)
Requirement already satisfied: charset-normalizer==4.0.0 in /usr/local/lib/python3.11/dist-packages (from requests==3.0.0->langchain_community<0.3.0) (3.4.2)
Requirement already satisfied: urllib3==3.0.0 in /usr/local/lib/python3.11/dist-packages (from requests==3.0.0->langchain_community<0.3.0) (2.4.0)

```

9. To install MLflow, select the **Run** icon in the second cell, as shown in the following screenshot.

```

!pip install mlflow
!pip install replicate
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy

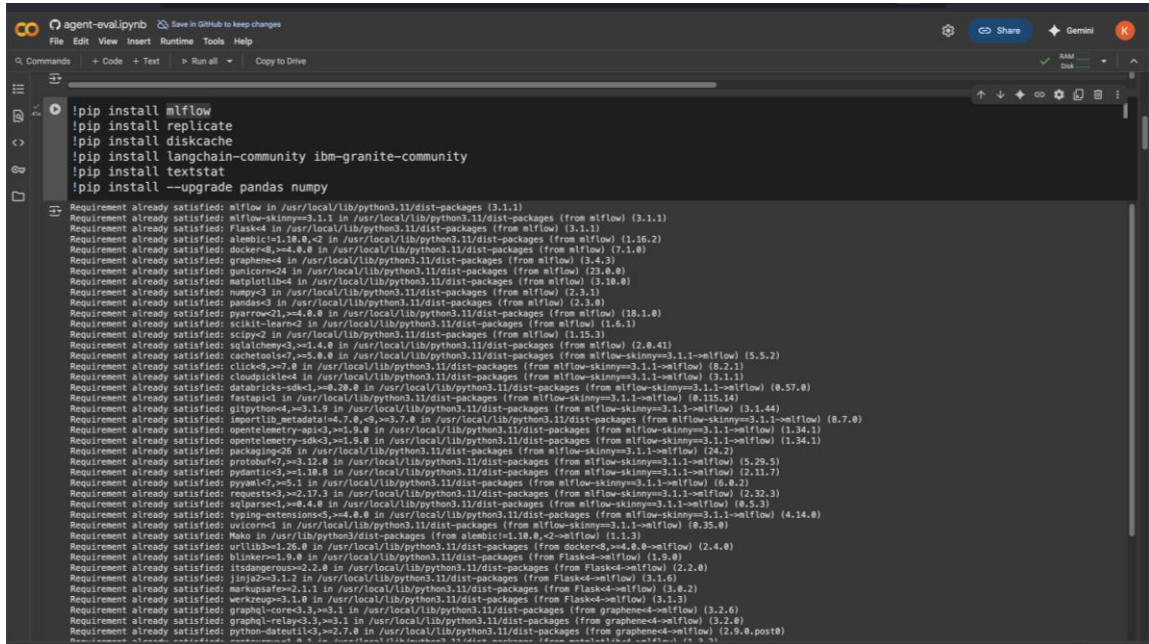
import mlflow
import pandas as pd
from ibm_granite_community.notebook_utils import get_env_var
from langchain_community.llms import Replicate
from typing import Dict, List, Any, Optional
import json
import re

model = Replicate(
    model="ibm-granite/granite-3.3-8b-instruct",
    replicate_api_token=get_env_var('REPLICATE_API_TOKEN'),
    model_kwargs={"max_tokens":1024, "temperature":0.2},
)

print('Setup complete!')

```

A green checkmark is displayed next to the **Run** icon, confirming that MLflow is successfully installed.

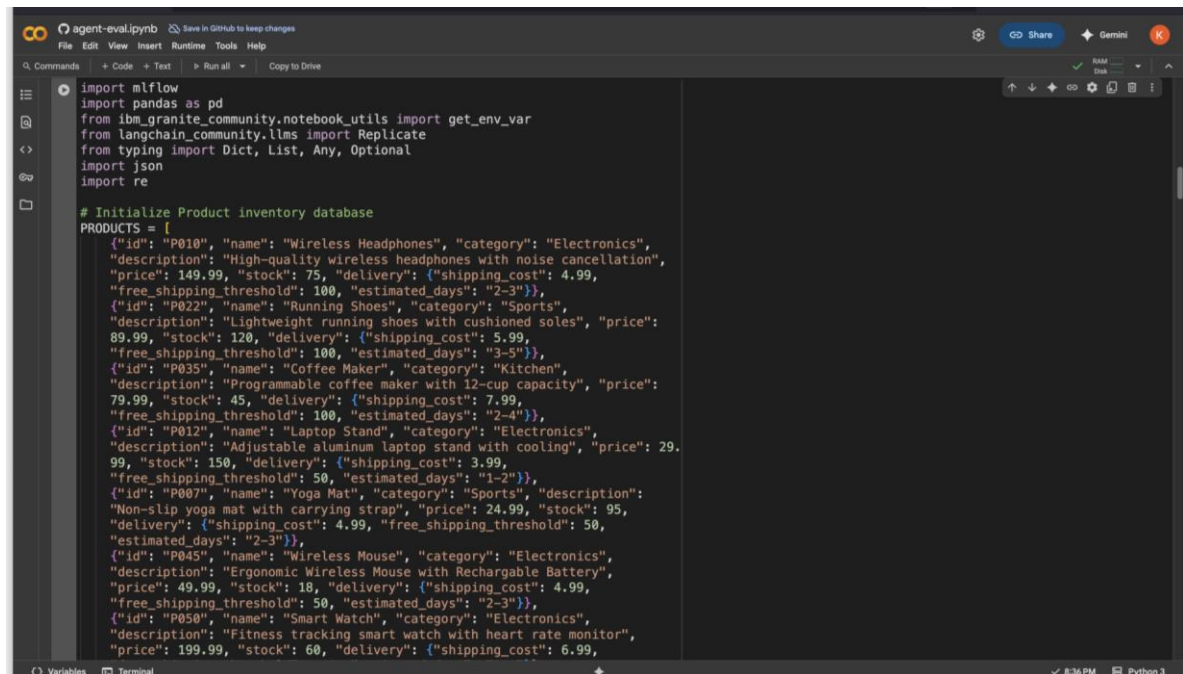


```
agent-eval.ipynb Save in GitHub to keep changes
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all Copy to Drive

!pip install mlflow
!pip install replicate
!pip install diskcache
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy

Requirement already satisfied: mlflow in /usr/local/lib/python3.11/dist-packages (3.1.1)
Requirement already satisfied: mlflow-skinny==3.1.1 in /usr/local/lib/python3.11/dist-packages (from mlflow) (3.1.1)
Requirement already satisfied: Flask<4 in /usr/local/lib/python3.11/dist-packages (from mlflow) (3.1.1)
Requirement already satisfied: alembic<1.18.0,<2 in /usr/local/lib/python3.11/dist-packages (from mlflow) (1.16.2)
Requirement already satisfied: docker<4,>=4.8 in /usr/local/lib/python3.11/dist-packages (from mlflow) (7.1.0)
Requirement already satisfied: graphene<4 in /usr/local/lib/python3.11/dist-packages (from mlflow) (3.4.3)
Requirement already satisfied: gunicorn<24 in /usr/local/lib/python3.11/dist-packages (from mlflow) (23.8.0)
Requirement already satisfied: matplotlib<4 in /usr/local/lib/python3.11/dist-packages (from mlflow) (3.10.0)
Requirement already satisfied: numpy<3 in /usr/local/lib/python3.11/dist-packages (from mlflow) (2.3.1)
Requirement already satisfied: pandas<2 in /usr/local/lib/python3.11/dist-packages (from mlflow) (2.3.0)
Requirement already satisfied: pyarrow<21,>=4.0 in /usr/local/lib/python3.11/dist-packages (from mlflow) (18.1.0)
Requirement already satisfied: scikit-learn<2 in /usr/local/lib/python3.11/dist-packages (from mlflow) (1.6.1)
Requirement already satisfied: scipy<2 in /usr/local/lib/python3.11/dist-packages (from mlflow) (1.15.3)
Requirement already satisfied: sqlalchemy<2,>=1.4.0 in /usr/local/lib/python3.11/dist-packages (from mlflow) (2.0.41)
Requirement already satisfied: cachetools<7,>=5.0.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (5.5.2)
Requirement already satisfied: click<9,>=7.8 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (8.2.1)
Requirement already satisfied: cloudpickle<4 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (3.1.1)
Requirement already satisfied: databricks-sdk<1,>=0.28.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (0.57.0)
Requirement already satisfied: fastapi<2 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (0.115.14)
Requirement already satisfied: importlib_metadata<4.7.0,<5,>=3.7.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (8.7.0)
Requirement already satisfied: opentelemetry-api<3,>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (1.34.1)
Requirement already satisfied: opentelemetry-sdk<3,>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (1.34.1)
Requirement already satisfied: packaging<26 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (24.2)
Requirement already satisfied: python-dateutil<3,>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (5.20.5)
Requirement already satisfied: pydantic<3,>=1.10.8 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (2.11.7)
Requirement already satisfied: pyyaml<7,>=5.1 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (6.0.2)
Requirement already satisfied: requests<3,>=2.2.3 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (2.32.3)
Requirement already satisfied: sqlalchemy<3,>=1.4.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (2.0.41)
Requirement already satisfied: typing-extensions<5,>=4.8.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (4.14.0)
Requirement already satisfied: Make in /usr/local/lib/python3.11/dist-packages (from alembic<1.18.0,<2->mlflow) (1.1.3)
Requirement already satisfied: urllib3<1.26.0 in /usr/local/lib/python3.11/dist-packages (from docker<8,>=4.0->mlflow) (2.4.0)
Requirement already satisfied: aiohttp<3.9.0 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (1.9.8)
Requirement already satisfied: itsdangerous<2.2.0 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (2.2.0)
Requirement already satisfied: Jinja2<3.1.2 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (3.1.6)
Requirement already satisfied: MarkupSafe<2.1.2 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (3.0.2)
Requirement already satisfied: Werkzeug<3.1.0 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (3.1.3)
Requirement already satisfied: graphql-core<3,>=3.1 in /usr/local/lib/python3.11/dist-packages (from graphene<4->mlflow) (3.2.6)
Requirement already satisfied: graphql-relay<3,>=3.1 in /usr/local/lib/python3.11/dist-packages (from graphene<4->mlflow) (3.2.0)
Requirement already satisfied: python-dateutil<3,>=2.7.0 in /usr/local/lib/python3.11/dist-packages (from graphene<4->mlflow) (2.9.0.post0)
```

10. To create a database for TechMart's product inventory, select the **Run** icon in the first cell of Task 2, as shown in the following screenshot.

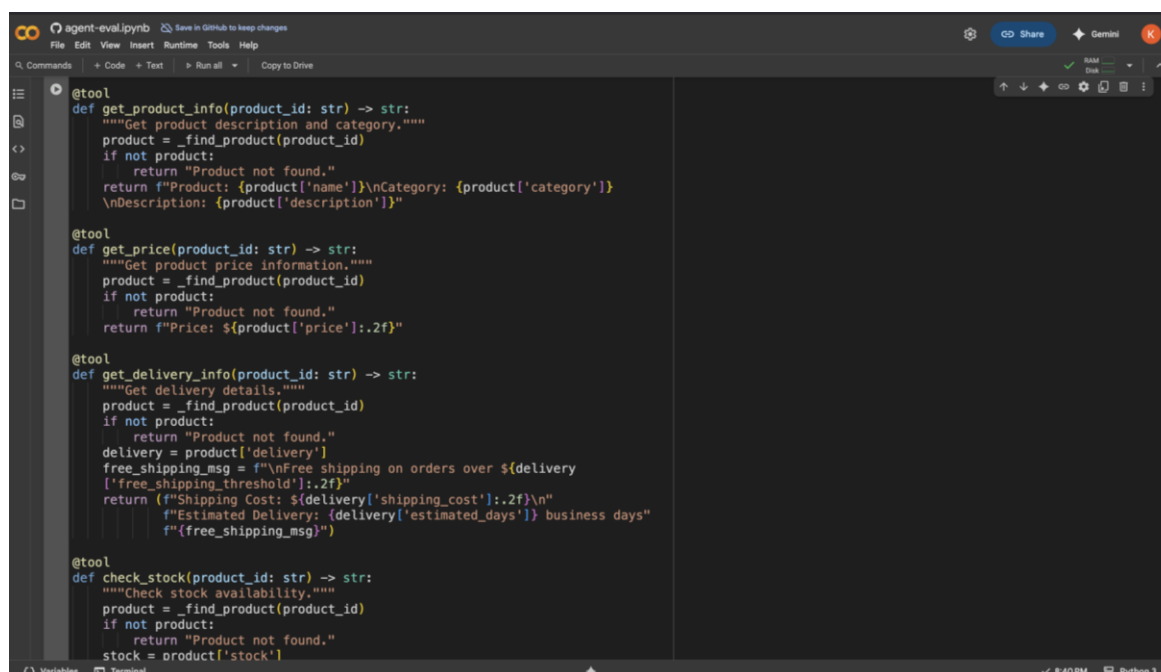


```
agent-eval.ipynb Save in GitHub to keep changes
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all Copy to Drive

import mlflow
import pandas as pd
from ibm_granite_community.notebook_utils import get_env_var
from langchain_community.llms import Replicate
from typing import Dict, List, Any, Optional
import json
import re

# Initialize Product inventory database
PRODUCTS = [
    {"id": "P010", "name": "Wireless Headphones", "category": "Electronics",
     "description": "High-quality wireless headphones with noise cancellation",
     "price": 149.99, "stock": 75, "delivery": {"shipping_cost": 4.99,
     "free_shipping_threshold": 100, "estimated_days": "2-3"}},
    {"id": "P022", "name": "Running Shoes", "category": "Sports",
     "description": "Lightweight running shoes with cushioned soles", "price":
     89.99, "stock": 120, "delivery": {"shipping_cost": 5.99,
     "free_shipping_threshold": 100, "estimated_days": "3-5"}},
    {"id": "P035", "name": "Coffee Maker", "category": "Kitchen",
     "description": "Programmable coffee maker with 12-cup capacity", "price":
     79.99, "stock": 45, "delivery": {"shipping_cost": 7.99,
     "free_shipping_threshold": 100, "estimated_days": "2-4"}},
    {"id": "P012", "name": "Laptop Stand", "category": "Electronics",
     "description": "Adjustable aluminum laptop stand with cooling", "price": 29.
     99, "stock": 150, "delivery": {"shipping_cost": 3.99,
     "free_shipping_threshold": 50, "estimated_days": "1-2"}},
    {"id": "P007", "name": "Yoga Mat", "category": "Sports", "description":
     "Non-slip yoga mat with carrying strap", "price": 24.99, "stock": 95,
     "delivery": {"shipping_cost": 4.99, "free_shipping_threshold": 50,
     "estimated_days": "2-3"}},
    {"id": "P045", "name": "Wireless Mouse", "category": "Electronics",
     "description": "Ergonomic Wireless Mouse with Rechargeable Battery",
     "price": 49.99, "stock": 18, "delivery": {"shipping_cost": 4.99,
     "free_shipping_threshold": 50, "estimated_days": "2-3"}},
    {"id": "P050", "name": "Smart Watch", "category": "Electronics",
     "description": "Fitness tracking smart watch with heart rate monitor",
     "price": 199.99, "stock": 60, "delivery": {"shipping_cost": 6.99,
```

11. To enable the agent to process user queries and generate meaningful responses, define the required tools such as **get_product_info**, **get_price**, **get_delivery_info**, and **check_stock**, as shown in the following screenshot.



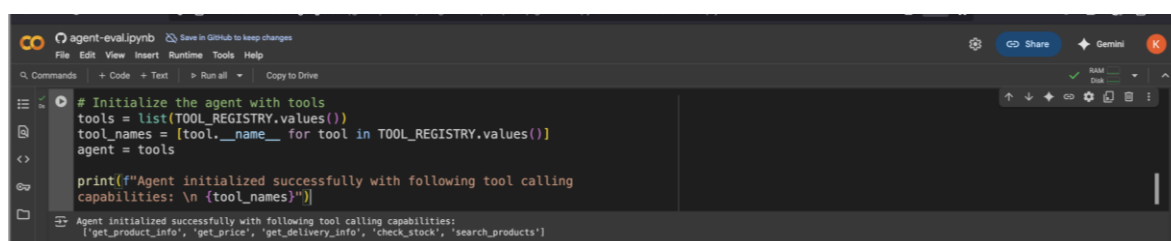
```
@tool
def get_product_info(product_id: str) -> str:
    """Get product description and category."""
    product = _find_product(product_id)
    if not product:
        return "Product not found."
    return f"Product: {product['name']}\nCategory: {product['category']}\nDescription: {product['description']}"

@tool
def get_price(product_id: str) -> str:
    """Get product price information."""
    product = _find_product(product_id)
    if not product:
        return "Product not found."
    return f"Price: ${product['price']:.2f}"

@tool
def get_delivery_info(product_id: str) -> str:
    """Get delivery details."""
    product = _find_product(product_id)
    if not product:
        return "Product not found."
    delivery = product['delivery']
    free_shipping_msg = f"\nFree shipping on orders over ${delivery['free_shipping_threshold']:.2f}"
    return (f"Shipping Cost: ${delivery['shipping_cost']:.2f}\n"
            f"Estimated Delivery: {delivery['estimated_days']} business days"
            f"{free_shipping_msg}")

@tool
def check_stock(product_id: str) -> str:
    """Check stock availability."""
    product = _find_product(product_id)
    if not product:
        return "Product not found."
    stock = product['stock']
```

12. To initialize the agent, select the **Run** icon, as shown in the following screenshot. The agent analyzes the user's input to identify whether it's asking for a product description or cost. Based on this, it uses the appropriate tools. For other types of input, the agent generates a general response.



```
# Initialize the agent with tools
tools = list(TOOL_REGISTRY.values())
tool_names = [tool.__name__ for tool in TOOL_REGISTRY.values()]
agent = tools

print(f"Agent initialized successfully with following tool calling capabilities: \n {tool_names}")

Agent initialized successfully with following tool calling capabilities:
['get_product_info', 'get_price', 'get_delivery_info', 'check_stock', 'search_products']
```

Evaluate the trajectory and final response of the AI agent

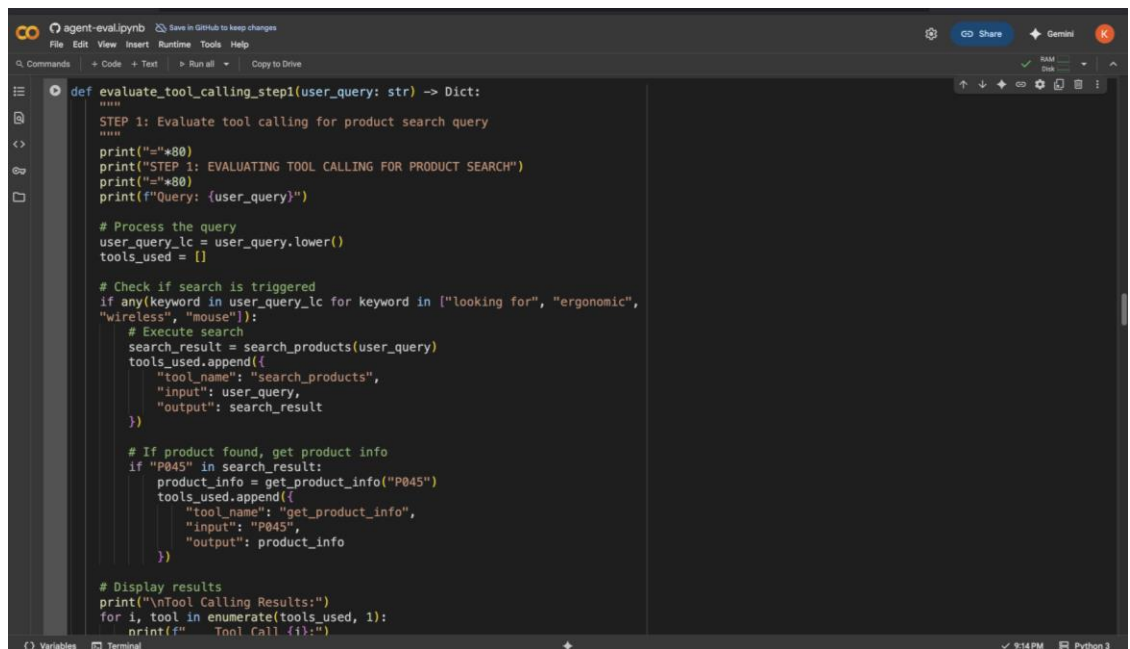
Overview

In this procedure, you'll evaluate the agent's trajectory and assess the accuracy, clarity, and relevance of its final responses using MLflow.

Instructions

1. To test how the agent responds, select the **Run** icon to invoke the **evaluation** functions.

Note: These evaluation functions captures and logs the agent's response, helping determine if the agent correctly analyzes the user's query, and invokes the appropriate tool. It validates whether the agent's behavior aligns with the expected context-driven tool usage and response generation.



```
def evaluate_tool_calling_step1(user_query: str) -> Dict:
    """
    STEP 1: Evaluate tool calling for product search query
    """
    print("="*80)
    print("STEP 1: EVALUATING TOOL CALLING FOR PRODUCT SEARCH")
    print("="*80)
    print(f"Query: {user_query}")

    # Process the query
    user_query_lc = user_query.lower()
    tools_used = []

    # Check if search is triggered
    if any(keyword in user_query_lc for keyword in ["looking for", "ergonomic",
    "wireless", "mouse"]):
        # Execute search
        search_result = search_products(user_query)
        tools_used.append({
            "tool_name": "search_products",
            "input": user_query,
            "output": search_result
        })

        # If product found, get product info
        if "P045" in search_result:
            product_info = get_product_info("P045")
            tools_used.append({
                "tool_name": "get_product_info",
                "input": "P045",
                "output": product_info
            })

    # Display results
    print("\nTool Calling Results:")
    for i, tool in enumerate(tools_used, 1):
        print(f"    Tool Call {i}:")
```

2. To test how the agent interprets a user query and uses the appropriate tool, select the **Run** icon to execute the code as shown in the following screenshot.

```
"""Run the complete lab exercise in steps"""  
print("MLflow Agent Evaluation Lab Exercise")  
print("="*80)  
  
# Step 1: Tool calling evaluation for product search  
query1 = "I'm looking for an ergonomic wireless mouse"  
step1_result = evaluate_tool_calling_step1(query1)
```

3. Review the tool-calling results to determine whether the agent correctly interprets the user query, "I'm looking for an ergonomic wireless mouse," and selects the right tools to generate a response.

```
MLflow Agent Evaluation Lab Exercise  
=====  
STEP 1: EVALUATING TOOL CALLING FOR PRODUCT SEARCH  
=====  
Query: I'm looking for an ergonomic wireless mouse  
  
Tool Calling Results:  
Tool Call 1:  
Tool Name : search_products  
Input      : I'm looking for an ergonomic wireless mouse  
Output     : Found the following products:  
- Wireless Mouse (ID: P045): Ergonomic Wireless Mouse with Rechargeable Battery  
  
Tool Call 2:  
Tool Name : get_product_info  
Input      : P045  
Output     : Product: Wireless Mouse  
Category: Electronics  
Description: Ergonomic Wireless Mouse with Rechargeable Battery  
  
Tool Calling Evaluation:  
✓ Search tool called: Yes  
✓ Product found: Yes  
✓ Correct product (P045): Yes
```

4. To evaluate whether the agent can handle follow-up questions by using multiple tools with the user's original query and context, select the **Run** icon to execute the code as shown in the screenshot.

```
# Step 2: Tool calling evaluation for stock query  
query2 = "Is the product available in stock?"  
step2_result = evaluate_tool_calling_step2(query2, "P045")
```

1. Review the output showing how the agent handles the query, "Is the product available in stock?" such as `get_product_info` and `check_inventory`, to generate a response based on the query.

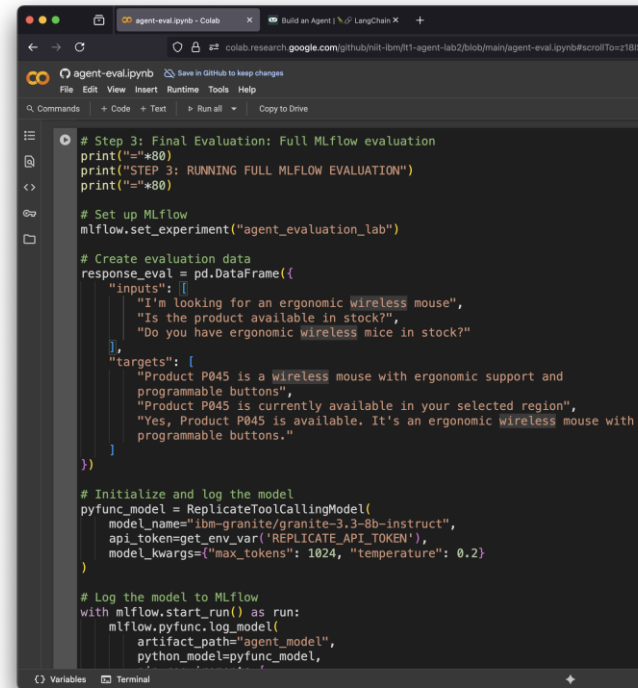
```
STEP 2: EVALUATING TOOL CALLING FOR STOCK QUERY  
=====  
Follow-up Query: Is the product available in stock?  
Product Context: P045  
  
Tool Calling Results:  
Tool Call 1:  
Tool Name : check_stock  
Input      : P045  
Output     : Stock Status: Limited Stock (18 units available)  
  
Tool Call 2:  
Tool Name : get_product_info  
Input      : P045  
Output     : Product: Wireless Mouse  
Category: Electronics  
Description: Ergonomic Wireless Mouse with Rechargeable Battery  
  
Stock Query Evaluation:  
✓ Stock tool called: Yes  
✓ Stock information provided: Yes  
✓ Correct product context: Yes
```

2. For the final agent evaluation, select the **Run** icon.

Evaluate the performance of an AI agent using MLflow

IBM SkillsBuild

5. For the final agent evaluation, select the R



```
# Step 3: Final Evaluation: Full MLflow evaluation
print("="*80)
print("STEP 3: RUNNING FULL MLFLOW EVALUATION")
print("="*80)

# Set up MLflow
mlflow.set_experiment("agent_evaluation_lab")

# Create evaluation data
response_eval = pd.DataFrame({
    "inputs": [
        "I'm looking for an ergonomic wireless mouse",
        "Is the product available in stock?",
        "Do you have ergonomic wireless mice in stock?"
    ],
    "targets": [
        "Product P045 is a wireless mouse with ergonomic support and programmable buttons",
        "Product P045 is currently available in your selected region",
        "Yes, Product P045 is available. It's an ergonomic wireless mouse with programmable buttons."
    ]
})

# Initialize and log the model
pyfunc_model = ReplicateToolCallingModel(
    model_name="ibm-granite/granite-3.3-8b-instruct",
    api_token=get_env_var('REPLICATE_API_TOKEN'),
    model_kwargs={"max_tokens": 1024, "temperature": 0.2}
)

# Log the model to MLflow
with mlflow.start_run() as run:
    mlflow.pyfunc.log_model(
        artifact_path="agent_model",
        python_model=pyfunc_model,
```

After the code is executed, the **response summary is displayed as shown in the following screenshot. Note that the agent can use tools such as get_product_info and check_inventory successfully and retrieve relevant information in response to user's query. Participants should then review the MLflow evaluation results, analyzing tool calling performance, response quality scores, and automated metrics to assess the agent's effectiveness in the e-commerce scenario.

Note: Even when an agent successfully retrieves relevant information using tools, its final response requires human evaluation for confirmation.



```
=====
Eval 1: Product Information
=====

Tool Calling Results:

Tool Call 1:
Tool Name : get_product_info
User Input : I'm looking for an ergonomic wireless mouse
Output : Product P045 is a wireless mouse with ergonomic support and programmable buttons.

Tool Call 2:
Tool Name : check_inventory
User Input : Is Product P045 available in stock?
Output : Product P045 is currently available in your selected region.
=====

Final Agent Response Summary:
=====
Query : Do you have ergonomic wireless mice in stock?
Tool Used : check_inventory
Response : Yes, Product P045 is available. It's an ergonomic wireless mouse with programmable buttons. Would you like to place an order?
Evaluation Score : 4/5
Feedback : The response is relevant and accurate, though it could be slightly more detailed about stock levels.
=====

🎉 Congratulations!

You've completed the simple AI agent evaluation lab! You've learned:
• How to work with a simple AI agent
• How to use different tools
• How to evaluate AI responses

Feel free to experiment with different queries and tools!
```

Conclusion

Congratulations! You've effectively evaluated TechMart's AI agent. Through structured testing and analysis with MLflow, you assessed the agent's tool-calling efficiency and the quality of its final responses, ensuring accurate, relevant, and clear delivery of product information.

© Copyright IBM Corporation 2025.

The information contained in these materials is provided for informational purposes only and is provided AS IS without warranty of any kind, express or implied. IBM shall not be responsible for any damages arising out of the use of, or otherwise related to, these materials. Nothing contained in these materials is intended to, nor shall have the effect of, creating any warranties or representations from IBM or its suppliers or licensors, or altering the terms and conditions of the applicable license agreement governing the use of IBM software. References in these materials to IBM products, programs, or services do not imply that they will be available in all countries in which IBM operates. This information is based on current IBM product plans and strategy, which are subject to change by IBM without notice. Product release dates and/or capabilities referenced in these materials may change at any time at IBM's sole discretion based on market opportunities or other factors, and are not intended to be a commitment to future product or feature availability in any way.

IBM, the IBM logo and ibm.com are trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at www.ibm.com/legal/copytrade.shtml.



Please Recycle